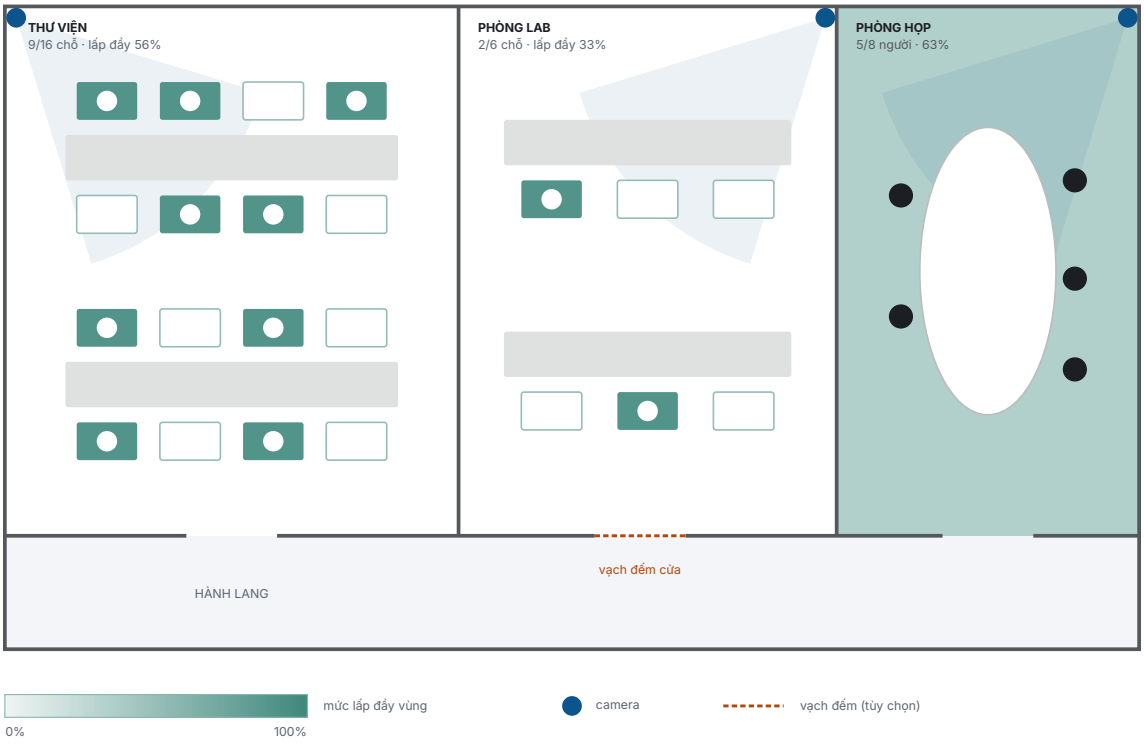
ĐẶC TẢ DỰ ÁN · COMPUTER VISION · PHIÊN BẢN 1.0 · 10/11/2026

Indoor Occupancy & Space Utilization System

Đo số người, mức lấp đầy và thời gian sử dụng từng vùng trong phòng làm việc, phòng thí nghiệm, thư viện: từ bài toán đến phần mềm triển khai được bất cứ lúc nào



Minh họa: mỗi chỗ ngồi ở thư viện và phòng lab là một vùng riêng; phòng họp được đếm theo cả phòng. Hệ thống chỉ lưu số đếm theo vùng, không lưu hình ảnh và không nhận dạng ai.

Phát triển từ: Real-Time Multi-Camera Video Analytics System, brief v2.0 (07/11/2026)

Công nghệ chính: YOLO + ONNX Runtime CUDA · ByteTrack · MediaMTX · FastAPI · SQLite · Docker Compose · GitHub Actions

Soạn cho: Tuan · **Người soạn:** Claude (Anthropic) · **Trạng thái:** bản nháp để rà soát

Ngưỡng, tham số và mốc thời gian trong tài liệu là đề xuất ban đầu, sẽ chốt lại sau khi có số đo baseline ở Giai đoạn 1. Số liệu có dấu * là ước tính. Phiên bản thư viện trong ví dụ cần đối chiếu bảng tương thích chính thức trước khi dùng.

Tóm tắt

Dự án trong một trang

Bài toán. Các tòa nhà, phòng lab và thư viện thường không biết chính xác không gian của mình được dùng bao nhiêu: phòng nào luôn đông, phòng nào bỏ trống, giờ nào cao điểm, sức chứa thực tế so với thiết kế. Đếm tay thì tốn công và chỉ có số liệu rời rạc; dữ liệu thẻ ra vào chỉ biết ai vào tòa nhà, không biết người ngồi ở đâu.
Giải pháp. Một phần mềm chạy tại chỗ (on-premise) trên một PC có GPU. Phần mềm đọc luồng video từ camera, phát hiện người bằng mô hình pretrained, đếm số người trong từng vùng (chỗ ngồi, dãy bàn, cả phòng), rồi tính mức lấp đầy, tỉ lệ thời gian sử dụng, giờ cao điểm và thời gian lưu trú. Kết quả có qua REST API, dashboard và file CSV. Hệ thống chỉ lưu số đếm; khung hình không bao giờ rời khỏi RAM.
Khác với bản brief v2.0. Trọng tâm chuyển từ "đếm lượt qua vạch thời gian thực" sang "chuỗi occupancy theo vùng". Người ngồi bị bàn che chân, đứng yên hàng giờ và thay đổi chậm, nên hệ thống lấy mẫu 1–2 FPS, gán người vào vùng theo tâm hoặc độ chồng lấn của bounding box, và tính phiên lưu trú theo trạng thái chỗ ngồi thay vì theo `track_id`.
Mục tiêu đo được (đề xuất). MAE số người theo vùng $\leq 0{,}5$ người; sai số mức lấp đầy theo giờ ≤ 5 điểm phần trăm; ≥ 8 camera ở 2 FPS trên một GPU; dashboard trễ ≤ 3 s (p95); tự phục hồi ≤ 10 s; chạy liên tục 24 giờ không lỗi; triển khai hoặc rollback bằng một lệnh.
Lộ trình. 9 giai đoạn trong khoảng 11 tuần. Giai đoạn 0 dựng ngay một "bộ khung chạy được" (walking skeleton) có CI, Docker và script deploy. Sau đó mỗi giai đoạn kết thúc bằng một bản phát hành có số hiệu, nên nhánh `main` lúc nào cũng deploy được.
Rủi ro lớn nhất. Mô hình pretrained có thể bỏ sót nhiều người ngồi bị che hoặc nhìn từ trên cao. Phép thử khả thi ở Giai đoạn 1 (mục 2.1) quyết định có tiếp tục với cấu hình hiện tại hay phải đổi góc camera hoặc mô hình.

Cách đọc tài liệu

Bạn cần	Đọc
Hiểu bài toán và mục tiêu	Chương 1 (bài toán, chỉ số, phạm vi) và Chương 2 (thách thức đặc thù)
Thiết kế và viết mã	Chương 3 (kiến trúc), 4 (thuật toán), 5 (đặc tả chức năng)
Lập kế hoạch làm việc	Chương 7 (lộ trình 9 giai đoạn) và Chương 8 (CI/CD, phát hành)
Kiểm thử và báo cáo số đo	Chương 9
Cài đặt tại một địa điểm thật	Chương 10 (triển khai, vận hành) và Chương 11 (quyền riêng tư)
Bắt đầu repo	Phụ lục A đến E (cấu trúc repo, Compose, Dockerfile, Makefile, CI)

Mục lục

Tóm tắt i

1 Bài toán và mục tiêu 1

1.1 Bối cảnh . . . 1
1.2 Câu hỏi hệ thống phải trả lời . . . 1
1.3 Người dùng và kịch bản sử dụng . . . 1
1.4 Thuật ngữ và định nghĩa chỉ số . . . 2
1.5 Mục tiêu đo được . . . 2
1.6 Phạm vi . . . 3
1.7 Thay đổi so với brief v2.0 . . . 4

2 Thách thức kỹ thuật đặc thù 5

2.1 Phép thử khả thi (go/no-go) . . . 5

3 Kiến trúc hệ thống 7

3.1 Sơ đồ tổng quan . . . 7
3.2 Thành phần và trách nhiệm . . . 7
3.3 Các quyết định thiết kế . . . 8
3.4 Dòng thời gian của một mẫu . . . 9
3.5 Môi trường . . . 9

4 Thuật toán analytics 10

4.1 Lọc detection . . . 10
4.2 Gán người vào vùng . . . 10
4.3 Làm mượt theo thời gian . . . 11
4.4 Trạng thái chỗ ngồi và phiên . . . 11
4.5 Phiên theo track trong vùng mở . . . 11
4.6 Đếm qua cửa (tùy chọn) . . . 12
4.7 Tổng hợp theo thời gian . . . 12
4.8 Tham số mặc định . . . 12

5 Đặc tả chức năng 14

5.1 Yêu cầu chức năng và tiêu chí chấp nhận . . . 14
5.2 Cấu hình . . . 15
5.3 Mô hình dữ liệu . . . 16
5.4 REST API . . . 17

5.5 Dashboard . . . 18
5.6 Công cụ vẽ vùng . . . 18

6 Yêu cầu phi chức năng 19
6.1 Cấu hình chuẩn để so sánh . . . 19
6.2 Bảng yêu cầu . . . 19
6.3 Metric và log . . . 20

7 Lộ trình từ A đến Z 21
7.1 Nguyên tắc . . . 21
7.2 Tổng quan tiến độ . . . 21
7.3 Dự phòng và phiên bản rút gọn . . . 25

8 Quy trình phát triển và phát hành liên tục 26
8.1 Nhánh và commit . . . 26
8.2 Pipeline CI/CD . . . 26
8.3 Phiên bản và tương thích . . . 27
8.4 Cờ tính năng . . . 27
8.5 Danh sách kiểm trước khi phát hành . . . 27
8.6 Cài đặt, nâng cấp, rollback . . . 27

9 Kiểm thử và đánh giá 29
9.1 Các tầng kiểm thử . . . 29
9.2 Bộ dữ liệu và giao thức gán nhãn . . . 30
9.3 Chỉ số đánh giá . . . 30
9.4 Giao thức benchmark . . . 30
9.5 Kiểm thử bơm lỗi . . . 31
9.6 Soak test . . . 31
9.7 Kiểm thử quyền riêng tư . . . 31
9.8 Nghiệm thu tại hiện trường . . . 31

10 Triển khai và vận hành 33
10.1 Yêu cầu phần cứng và phần mềm . . . 33
10.2 Mô hình mạng . . . 33
10.3 Các bước triển khai lần đầu . . . 33
10.4 Thay đổi thường gặp . . . 34
10.5 Giám sát và cảnh báo . . . 34
10.6 Sao lưu và phục hồi . . . 35
10.7 Runbook sự cố . . . 35

11 Quyền riêng tư, bảo mật và giấy phép **36**
11.1 Nguyên tắc . . . 36
11.2 Kiểm kê dữ liệu . . . 36
11.3 Thông báo và đồng ý . . . 36
11.4 Bảo mật . . . 37
11.5 Giấy phép . . . 37

12 Rủi ro và giảm thiểu **38**

13 Definition of Done cho v1.0 **39**

A Cấu trúc repository **40**

B File Compose mẫu **41**

C Dockerfile của engine **43**

D Makefile **44**

E Workflow CI **45**

F Tài liệu tham khảo **47**

CHƯƠNG 1

Bài toán và mục tiêu

1.1 Bối cảnh

Không gian trong nhà là một trong những chi phí lớn nhất của trường học, doanh nghiệp và thư viện, nhưng mức sử dụng thực tế thường chỉ được ước đoán. Có ba câu hỏi người quản lý cơ sở vật chất hay gặp: phòng nào thường xuyên quá tải, phòng nào bị lãng phí, và nên mở thêm hay thu hẹp không gian nào. Người dùng cuối thì có câu hỏi đơn giản hơn: "bây giờ thư viện còn chỗ không?"

Các cách đo hiện có đều có hạn chế. Đếm tay theo đợt tốn công và chỉ cho số liệu rời rạc. Dữ liệu thẻ ra vào hay Wi-Fi cho biết có người trong tòa nhà, nhưng không cho biết họ ngồi ở đâu. Cảm biến hồng ngoại đếm qua cửa bị trôi sai số theo thời gian. Camera kết hợp thị giác máy tính có thể đếm trực tiếp số người trong từng vùng, với điều kiện xử lý tốt vấn đề che khuất và quyền riêng tư.

1.2 Câu hỏi hệ thống phải trả lời

1. **Hiện tại:** mỗi vùng đang có bao nhiêu người, còn bao nhiêu chỗ trống? Cập nhật trong vài giây.
2. **Mức sử dụng:** trong một khoảng thời gian, mỗi vùng được lấp đầy trung bình bao nhiêu phần trăm sức chứa, và có người trong bao nhiêu phần trăm giờ mở cửa?
3. **Cao điểm:** giờ nào trong ngày, ngày nào trong tuần đông nhất? Số người cực đại là bao nhiêu?
4. **Lưu trú:** một chỗ ngồi thường được dùng liên tục bao lâu? Phân bố thời lượng các phiên ra sao?
5. **Xu hướng:** mức sử dụng thay đổi thế nào theo tuần, theo học kỳ, sau khi bố trí lại phòng?

1.3 Người dùng và kịch bản sử dụng

Vai trò	Nhu cầu	Chức năng tương ứng
Quản lý cơ sở vật chất	Báo cáo mức sử dụng theo tuần, so sánh các phòng, tìm phòng lãng phí	Dashboard lịch sử, bản đồ nhiệt theo giờ, xuất CSV
Người dùng cuối	Biết còn chỗ trống trước khi đến	Trang trạng thái trực tiếp chỉ đọc (số chỗ trống theo phòng)
Kỹ thuật viên	Lắp camera, vẽ vùng, theo dõi sức khỏe hệ thống	Công cụ vẽ vùng, trang trạng thái camera, cảnh báo, log
Nhà phát triển	Thêm tính năng mà không làm hỏng bản đang chạy	CI, bộ test hồi quy độ chính xác, môi trường mô phỏng

1.4 Thuật ngữ và định nghĩa chỉ số

Mọi chỉ số trong tài liệu được tính từ các định nghĩa dưới đây. Việc định nghĩa trước là bắt buộc: nếu không, "mức sử dụng 60%" có thể mang nhiều nghĩa khác nhau và không so sánh được.

Thuật ngữ	Định nghĩa
Vùng z (zone)	Một đa giác trên ảnh của một camera, gắn với một không gian thật: một chỗ ngồi, một dãy bàn hoặc cả phòng. Vùng thuộc một *nhóm* (ví dụ `seats`, `room`); trong cùng nhóm, mỗi người chỉ thuộc tối đa một vùng.
Sức chứa C_z	Số chỗ thiết kế của vùng, khai báo trong cấu hình.
Mẫu k	Một khung hình được lấy để xử lý tại thời điểm t_k (đồng hồ đơn điệu). Tần số lấy mẫu f_s cấu hình theo camera.
Số đếm thô $c_z(k)$	Số detection được gán vào vùng z ở mẫu k, sau khi lọc (mục 4.2).
Occupancy $\hat{n}_z(k)$	Số đếm đã làm mượt bằng trung vị trượt trên W mẫu gần nhất (mục 4.3).
Giờ mở cửa $\mathcal{T}$	Tập khoảng thời gian vùng được phép sử dụng, khai báo theo ngày trong tuần và ngày nghỉ. Chỉ số sử dụng chỉ tính trong $\mathcal{T}$.
Phiên (session)	Một khoảng thời gian liên tục một chỗ ngồi có người, hoặc một người ở trong một vùng mở.

Gọi K_T là tập các mẫu trong khoảng $T \subseteq \mathcal{T}$ và Δt là khoảng cách giữa hai mẫu. Các chỉ số chính:

$$\text{Mức lấp đầy} \quad F_z(T) = \frac{1}{|K_T|} \sum_{k \in K_T} \frac{\hat{n}_z(k)}{C_z} \tag{1.1}$$

$$\text{Tỉ lệ thời gian sử dụng} \quad U_z(T) = \frac{1}{|K_T|} \sum_{k \in K_T} \mathbf{1}[\hat{n}_z(k) \geq 1] \tag{1.2}$$

$$\text{Cao điểm} \quad P_z(T) = \max_{k \in K_T} \hat{n}_z(k), \qquad P_z^{95}(T) = \text{p95}_{k \in K_T}\, \hat{n}_z(k) \tag{1.3}$$

$$\text{Chỗ-giờ (seat-hours)} \quad H_z(T) = \frac{\Delta t}{3600} \sum_{k \in K_T} \hat{n}_z(k) \tag{1.4}$$

F_z trả lời câu hỏi "dùng bao nhiêu phần trăm sức chứa"; U_z trả lời "có người trong bao nhiêu phần trăm thời gian". Một phòng họp 10 chỗ thường chỉ có 2 người họp suốt ngày sẽ có U_z cao nhưng F_z thấp, tức là phòng quá lớn so với nhu cầu. Đây chính là loại phát hiện mà người quản lý cần. P_z^{95} được báo cáo kèm P_z vì giá trị cực đại dễ bị một mẫu nhiễu làm sai.

> **Quyết định định nghĩa: chỉ đếm người nhìn thấy được**
> Một người đi ra ngoài để lại cặp sách trên ghế: hệ thống **không** tính ghế đó là đang dùng sau khi hết thời gian ân hạn (mục 4.4). Phát hiện "giữ chỗ bằng đồ vật" là bài toán khác, cần phát hiện đồ vật, và nằm ngoài phạm vi v1.0. README và báo cáo phải ghi rõ định nghĩa này.

1.5 Mục tiêu đo được

Các ngưỡng là đề xuất ban đầu và sẽ chốt lại sau số đo baseline ở Giai đoạn 1. Nếu cuối dự án chưa đạt, báo cáo phải giải thích bằng số đo thay vì bỏ qua.

Mã	Mục tiêu	Định nghĩa đo	Ngưỡng đề xuất
M1	Độ chính xác đếm	MAE giữa $\hat{n}_z$ và nhãn tay trên các mẫu có nhãn của tập test (mục 9.3)	$\leq$ max$(0{,}5;\ 0{,}05\, C_z)$ người
M2	Độ chính xác chỉ số sử dụng	Sai số tuyệt đối của F_z và U_z tính theo từng giờ	≤ 5 điểm phần trăm

Mã	Mục tiêu	Định nghĩa đo	Ngưỡng đề xuất
M3	Phiên chỗ ngồi	Tỉ lệ phiên thật được khớp; sai số thời điểm bắt đầu/kết thúc; tỉ lệ phân mảnh	$\geq 90\%$; ≤ 30 s; $\leq 1{,}2$
M4	Quy mô	Số camera tối đa trên một GPU ở $f_s = 2$ FPS mà tỉ lệ mẫu bị bỏ $\leq 1\%$	≥ 8 camera
M5	Độ tươi dữ liệu	Từ lúc nhận khung đến lúc dashboard hiển thị số mới, p95	≤ 3 s
M6	Phục hồi	Từ lúc nguồn video sống lại đến lúc camera về `ONLINE`; camera khác không bị ảnh hưởng	≤ 10 s
M7	Ổn định	Soak test liên tục; RAM tăng so với sau giờ đầu; số lần crash	24 giờ; $< 10\%$; 0
M8	Triển khai	Từ máy đã cài driver và Docker đến hệ thống chạy; nâng cấp hoặc rollback	≤ 15 phút, 1 lệnh; ≤ 5 phút, không mất dữ liệu
M9	Quyền riêng tư	Số file ảnh hoặc video được ghi ra đĩa ở chế độ production (kiểm thử tự động)	0

1.6 Phạm vi

Trong phạm vi v1.0	Ngoài phạm vi (phiên bản sau)
Camera IP qua RTSP; webcam USB và file MP4 qua FFmpeg đẩy lên MediaMTX	Nhận dạng khuôn mặt hoặc bất kỳ thông tin định danh nào
Phát hiện người bằng mô hình pretrained (YOLO, ONNX Runtime CUDA; chạy được trên CPU cho môi trường thử)	Nhận dạng lại người giữa các camera (cross-camera Re-ID)
Vùng đa giác theo nhóm, vùng loại trừ, sức chứa; occupancy làm mượt	Phát hiện giữ chỗ bằng đồ vật
Phiên theo chỗ ngồi; phiên theo track có nối track trong vùng mở	Quy đổi sang tọa độ sàn bằng homography
Đếm qua cửa (tính năng tùy chọn, bật bằng cờ cấu hình)	Fine-tune mô hình (chỉ làm nếu phép thử khả thi thất bại, xem mục 2.1)
Tổng hợp theo phút, giờ, ngày; giờ mở cửa; chính sách lưu giữ dữ liệu	Đăng nhập một lần (SSO), nhiều tổ chức trên một hệ thống (multi-tenant)
REST API có phiên bản, WebSocket trạng thái trực tiếp, xuất CSV	Triển khai trên cloud, ứng dụng di động
Dashboard, công cụ vẽ vùng, trang trạng thái công khai chỉ đọc	Lưu trữ video
Xác thực đơn giản bằng token và mật khẩu quản trị; HTTPS qua reverse proxy	Tích hợp hệ thống đặt phòng, BMS của tòa nhà
Log JSON, metric Prometheus, cảnh báo, sao lưu, nâng cấp và rollback	

Brief v2.0	Bản này	Lý do

1.7 Thay đổi so với brief v2.0

Brief v2.0	Bản này	Lý do
Đếm qua vạch là chỉ số chính	Occupancy theo vùng là chính; đếm cửa là tùy chọn	Bài toán hỏi "bao nhiêu người trong vùng", không phải "bao nhiêu lượt qua cửa"
25 FPS, p95 latency ≤ 300 ms	1–2 FPS cho camera trong phòng, 10 FPS cho camera cửa; đo số camera trên GPU	Occupancy thay đổi chậm; con số "bao nhiêu camera trên một GPU" có giá trị triển khai hơn
Điểm neo ở giữa cạnh dưới bbox	Điểm neo cấu hình được (mặc định tâm) hoặc tỉ lệ chồng lấn	Người ngồi bị bàn che chân
Dwell theo `track_id`, ân hạn 1,5 s	Phiên theo trạng thái chỗ ngồi, ân hạn 60 s; nối track trong vùng mở	Người ngồi lâu làm tracker đổi ID nhiều lần
Một tiến trình GPU cho mỗi camera	Một tiến trình đọc cho mỗi camera, một tiến trình suy luận gộp batch	Tránh mỗi camera chiếm một CUDA context; dễ mở rộng lên nhiều camera
Backoff tối đa 30 s	Backoff tối đa 5 s, timeout đọc 5 s	Mâu thuẫn với mục tiêu phục hồi ≤ 10 s
Sai số đếm $\lvert\hat{n} - n\rvert / n$	MAE theo mẫu, sai số F_z, U_z, chỉ số khớp phiên	Sai số tổng che được lỗi đếm thừa và thiếu bù trừ nhau
Soak test 2 giờ	Soak test 24 giờ trước mỗi bản phát hành; pilot 7 ngày trước v1.0	Phần mềm phải deploy được, không chỉ demo được
Không có xác thực	Token API, mật khẩu quản trị, HTTPS tùy chọn	Hệ thống chạy trong mạng của tổ chức khác

CHƯƠNG 2

Thách thức kỹ thuật đặc thù

Phần lớn pipeline giống một hệ thống đếm người thông thường. Độ khó thật nằm ở chỗ người trong phòng làm việc và thư viện *ngồi, đứng yên lâu và bị che nhiều*, trong khi mô hình pretrained chủ yếu được huấn luyện trên ảnh người đứng hoặc đi lại. Bảng dưới đây liệt kê các thách thức và nơi tài liệu xử lý chúng.

Thách thức	Hậu quả nếu bỏ qua	Hướng giải quyết	Mục
Người ngồi bị bàn, màn hình, ghế che	Detector bỏ sót; điểm neo ở chân rơi ra ngoài vùng	Kiểm tra recall người ngồi ngay từ đầu; điểm neo ở tâm bbox hoặc gán theo độ chồng lấn; vẽ vùng theo khu bàn ghế	2.1, 4.2
Người đứng yên hàng giờ	Detection chớp tắt khi cúi xuống; tracker đổi ID; một lượt ngồi bị cắt thành nhiều lượt	Làm mượt theo thời gian; phiên theo trạng thái chỗ ngồi với ân hạn dài; nối track trong vùng mờ	4.3–4.5
Camera gắn trần nhìn xuống, ống kính góc rộng hoặc fisheye	Mô hình pretrained giảm recall rõ rệt ở góc nhìn từ trên	Chọn góc nghiêng thay vì thẳng đứng; thử mô hình lớn hơn và `imgsz` lớn hơn; dự phòng mô hình phát hiện đầu người	2.1
Người trên poster, màn hình chiếu, gương, kính	Đếm thừa cố định, rất khó phát hiện bằng mắt trên số liệu tổng	Vùng loại trừ (exclusion mask) trong cấu hình; kiểm tra bằng clip phòng trống	4.2
Mật độ cao, người che nhau	Đếm thiếu ở giờ cao điểm, đúng lúc số liệu quan trọng nhất	Báo cáo sai số tách theo mức đông; ghi giới hạn trong README; ưu tiên góc nhìn cao	9.3
Ánh sáng thay đổi theo giờ, đèn tắt bật	Sai số khác nhau giữa sáng và tối; thiên lệch theo giờ	Bộ test phải có cả ánh sáng ban ngày, đèn và chập tối	9.2
Lấy mẫu thưa (1–2 FPS)	Tracker dựa trên chuyển động kém đi vì người dịch chuyển xa giữa hai mẫu	Occupancy không cần tracking; tracking chỉ dùng cho phiên trong vùng mờ và camera cửa ở 10 FPS	4.5
Giám sát nơi học và làm việc	Mất niềm tin của người bị quay; rủi ro pháp lý	Không ghi khung hình; chỉ lưu số đếm theo vùng; không lưu `track_id`; thông báo và xin đồng ý	11

2.1 Phép thử khả thi (go/no-go)

Trước khi xây phần analytics, cần biết detector có thấy đủ người ngồi ở góc camera thật hay không. Nếu không thấy, mọi thuật toán phía sau đều không cứu được. Phép thử này làm trong Giai đoạn 1 và chỉ tốn khoảng một đến hai ngày.

Cách làm.

1. Quay 15–30 phút ở đúng vị trí và góc camera dự định dùng, có người ngồi làm việc, có lúc đông lúc vắng (có sự đồng ý của người trong hình).
2. Trích một khung mỗi 15 s (khoảng 60–120 khung). Gán nhãn bounding box mọi người trong khung bằng CVAT, kèm thuộc tính `posture` (`standing`, `sitting`) và `occluded` (che dưới hoặc trên 50%).
3. Chạy detector hiện có với vài cấu hình: mô hình cỡ n và s, `imgsz` 640 và 960, `conf` 0,25 và 0,35.
4. Tính recall và precision ở IoU $\geq 0{,}5$, tách theo `posture` và `occluded`. Đếm số false positive cố định (poster, màn hình).

Quyết định. Dựa trên recall người ngồi của cấu hình tốt nhất:

Recall người ngồi (cấu hình tốt nhất)	Hành động
$\geq 0{,}90$	Tiếp tục với cấu hình này làm baseline.
$0{,}75$ – $0{,}90$	Tiếp tục, nhưng thử thêm mô hình cỡ m, `imgsz` 1280, hoặc chia ảnh thành ô (tiling) cho vùng xa; đánh giá lại chi phí GPU. Ghi giới hạn.
$< 0{,}75$	Dừng lại. Thử đổi góc hoặc vị trí camera trước. Nếu vẫn thấp, đưa "phát hiện đầu người" hoặc fine-tune vào phạm vi và giãn lộ trình 1–2 tuần.

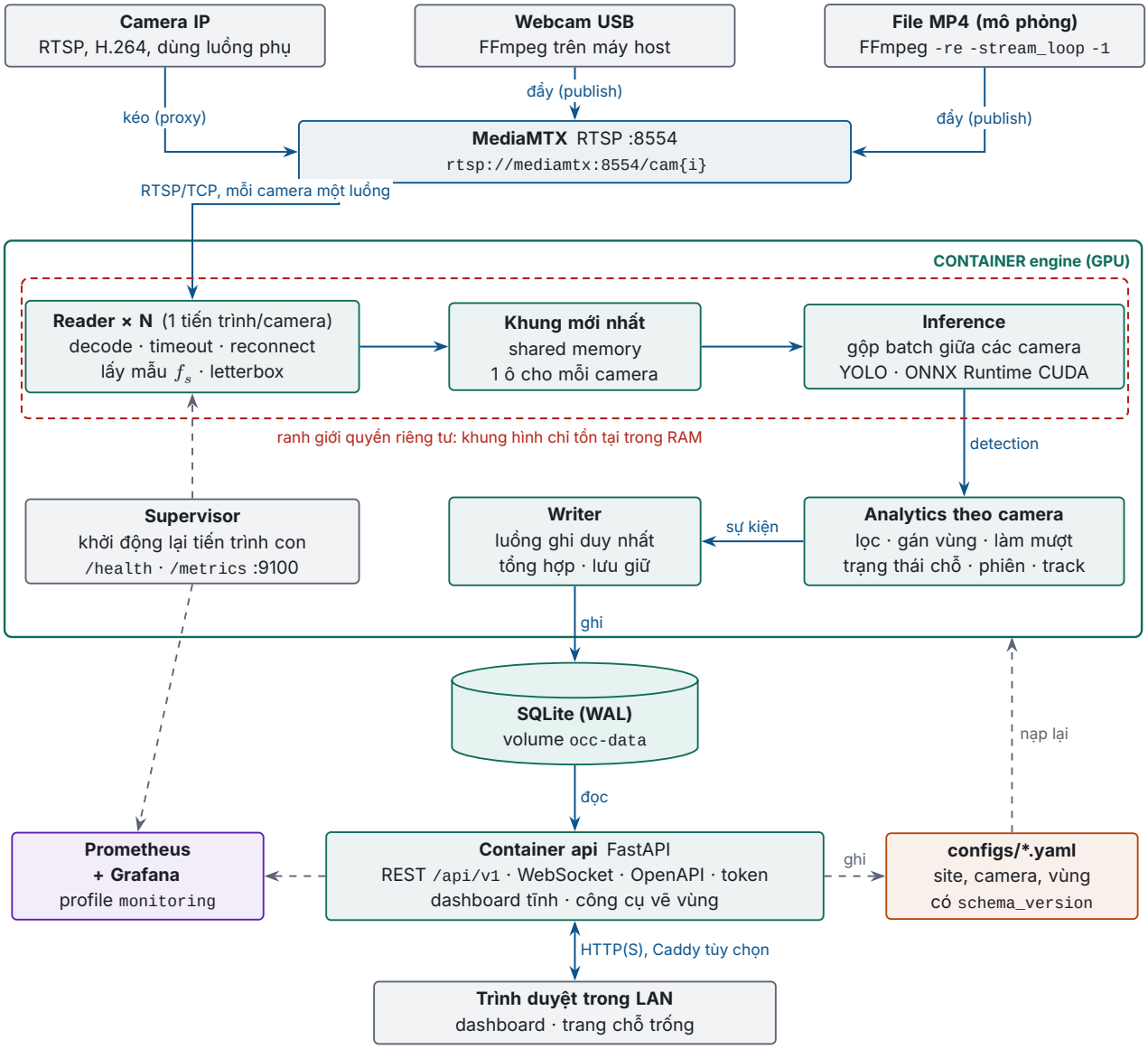
Vì sao làm phép thử trước

Bạn đã có code detect person, nên phép thử này gần như miễn phí. Nó trả lời câu hỏi rủi ro nhất của dự án trước khi bỏ công vào những phần phụ thuộc vào nó. Ngưỡng 0,90 và 0,75 là đề xuất: thay đổi được, nhưng phải chốt trước khi chạy để tránh tự điều chỉnh cho vừa kết quả.

CHƯƠNG 3

Kiến trúc hệ thống

3.1 Sơ đồ tổng quan



Hình 3.1: Kiến trúc tổng quan. Mũi tên liền là luồng dữ liệu; mũi tên đứt là điều khiển, cấu hình và metric.

3.2 Thành phần và trách nhiệm

Thành phần	Trách nhiệm	Công nghệ
Nguồn video	Camera IP được MediaMTX kéo về (proxy) để chỉ có một kết nối tới camera. Webcam và file MP4 được FFmpeg đẩy lên. Mọi reader chỉ đọc RTSP từ MediaMTX, nên chỉ có một kiểu nguồn cần xử lý.	MediaMTX, FFmpeg
Reader	Mỗi camera một tiến trình. Đọc RTSP qua TCP với timeout mở và đọc; decode liên tục để không dồn buffer; mỗi $1/f_s$ giây chép khung mới nhất (đã letterbox) vào ô shared memory kèm `camera_id`, số thứ tự, t_0 đơn điệu và thời gian UTC. Báo trạng thái `ONLINE`, `RECONNECTING`, `OFFLINE`.	OpenCV (backend FFmpeg) hoặc PyAV; `multiprocessing.shared_memory`
Inference	Gom các ô có khung mới thành một batch (tối đa B khung hoặc chờ tối đa 50 ms), chạy detector, trả detection về đúng camera. Chạy được trên CPU cho môi trường dev và CI.	ONNX Runtime (CUDA EP, CPU EP); YOLO xuất ONNX với batch động
Analytics	Trạng thái riêng cho từng camera: lọc detection, gán vào vùng, làm mượt, máy trạng thái chỗ ngồi, tracker và nối track, đếm cửa. Nhận đồng hồ qua tham số để chạy offline được.	Python, NumPy, Shapely; ByteTrack bản độc lập
Writer	Luồng ghi duy nhất vào SQLite: cập nhật bảng `zone_live` mỗi giây, ghi snapshot 10 s, tổng hợp theo phút, ghi phiên, chạy chính sách lưu giữ.	SQLite (WAL), SQLAlchemy Core, Alembic
Supervisor	Tiến trình gốc của container engine: khởi động và theo dõi tiến trình con, khởi động lại khi chết, phục vụ `/health` và `/metrics`.	Python, `prometheus_client`
API	REST có phiên bản, WebSocket trạng thái trực tiếp, xuất CSV, xác thực, phục vụ dashboard tĩnh và công cụ vẽ vùng. Chỉ đọc cơ sở dữ liệu; ghi cấu hình YAML khi quản trị viên sửa vùng.	FastAPI, Uvicorn, Pydantic
Dashboard	Trạng thái trực tiếp theo vùng, biểu đồ lịch sử, bản đồ nhiệt ngày × giờ, sức khỏe camera, trang chỗ trống công khai.	HTML, HTMX hoặc JS thuần, Chart.js
Giám sát	Thu metric, hiển thị và cảnh báo. Không bắt buộc để hệ thống chạy.	Prometheus, Grafana (profile)

3.3 Các quyết định thiết kế

D1. Một tiến trình đọc cho mỗi camera, một tiến trình suy luận chung. Mỗi camera có tiến trình reader riêng, nên luồng hỏng hoặc FFmpeg treo không kéo theo camera khác. Suy luận chạy trong một tiến trình duy nhất giữ mô hình trên GPU và gộp batch giữa các camera. Nếu mỗi camera có tiến trình GPU riêng, mỗi tiến trình sẽ có một CUDA context và một bản sao mô hình, chiếm vài trăm MB VRAM mỗi tiến trình*, nên số camera bị giới hạn bởi VRAM thay vì bởi khả năng tính. Trạng thái analytics vẫn tách hoàn toàn theo `camera_id`.

D2. Lấy mẫu thưa, nhưng decode liên tục. Camera trong phòng chỉ cần $f_s = 1$–2 FPS, nhưng reader vẫn phải đọc hết luồng, nếu không buffer RTSP sẽ dồn và khung lấy ra bị cũ. Để giảm tải CPU, cấu hình camera IP phát luồng phụ (substream) độ phân giải 720p và 5–10 FPS. Camera cửa dùng $f_s = 10$ FPS vì người đi lại nhanh.

D3. Đồng hồ là một tham số. Mọi tính toán thời gian dùng một đối tượng `Clock` truyền vào. Khi chạy thật dùng `time.monotonic_ns()` lấy tại thời điểm reader nhận khung; trên Linux đồng hồ này chung cho mọi tiến trình và container trên cùng máy. Khi đánh giá offline trên file, dùng `FrameClock` với $t = \text{frame_index}/\text{fps}$, nên kết quả tái lập được từng mẫu. Không dùng PTS của RTSP vì PTS đặt lại khi MP4 lặp.

D4. Ranh giới quyền riêng tư nằm trong kiến trúc. Khung hình chỉ tồn tại trong reader, shared memory và inference. Không có đường mã nào trong chế độ production ghi khung hình ra đĩa hoặc gửi qua mạng. Ngoại lệ duy nhất là ảnh chụp cho công cụ vẽ vùng (mục 5.6): làm mờ người trước khi gửi, chỉ giữ trong RAM và hết hạn sau 10 phút. Bảng cơ sở dữ liệu không có cột BLOB và không lưu `track_id`.

D5. Một tiến trình ghi duy nhất. SQLite ở chế độ WAL cho phép một tiến trình ghi và nhiều tiến trình đọc cùng lúc. Writer trong engine là nơi duy nhất ghi dữ liệu đo; API chỉ đọc. Lớp lưu trữ viết bằng SQLAlchemy Core để sau này đổi sang PostgreSQL mà không sửa logic. Volume dữ liệu phải là named volume trên ổ cục bộ, không phải thư mục chia sẻ mạng hay bind mount từ ổ Windows, vì WAL cần chia sẻ bộ nhớ giữa các tiến trình.

D6. Cấu hình là dữ liệu, có schema và phiên bản. Site, camera và vùng nằm trong file YAML có trường `schema_version`, được kiểm tra bằng Pydantic khi khởi động và bằng lệnh `occ config validate`. Cấu hình sai làm camera đó báo lỗi rõ ràng và dừng, không làm sập camera khác. Thay đổi vùng không cần khởi động lại: API ghi file, engine nạp lại cấu hình của đúng camera đó.

D7. Một image cho production không có PyTorch. Image production chỉ chứa ONNX Runtime, nên nhỏ hơn và khởi động nhanh hơn. PyTorch và Ultralytics nằm trong image `tools` dùng để xuất mô hình và benchmark. Tracker dùng một cài đặt ByteTrack độc lập (bản gốc hoặc thư viện supervision, đều MIT) để image production không phụ thuộc Ultralytics.

3.4 Dòng thời gian của một mẫu

Mốc	Sự kiện	Ghi chú
t_0	Reader nhận khung được chọn làm mẫu	Gắn `monotonic_ns` và UTC
t_1	Khung vào một batch	$t_1 - t_0$: thời gian chờ gom batch, tối đa 50 ms
t_2	Có detection	$t_2 - t_1$: suy luận GPU (gồm tiền xử lý)
t_3	Analytics cập nhật trạng thái vùng	Gán vùng, làm mượt, máy trạng thái
t_4	Writer commit `zone_live`	Gom theo chu kỳ 1 s, nên thêm tối đa 1 s
t_5	API đẩy giá trị mới qua WebSocket	API đọc `zone_live` mỗi giây

Độ tươi dữ liệu (M5) là $t_5 - t_0$, ước tính 1–2,5 s* với chu kỳ ghi và đọc 1 s. Bộ lọc trung vị W mẫu tạo thêm độ trễ *tín hiệu* khoảng $(W-1)/2$ mẫu, tức 1 s với $W = 5$ và $f_s = 2$. Độ trễ này là chủ ý (đổi lấy ổn định) và được báo cáo riêng, không tính vào M5.

3.5 Môi trường

Môi trường	Phần cứng và nguồn	Mục đích
dev	Laptop; backend CPU; camera mô phỏng từ MP4 (profile `sim`)	Viết mã, test nhanh, chạy toàn hệ thống không cần GPU
CI	Runner GitHub; CPU; MediaMTX và publisher trong job	Lint, test, hồi quy độ chính xác, smoke test toàn hệ thống
staging	PC có GPU; mô phỏng + webcam + 1 camera thật	Benchmark, bơm lỗi, soak 24 h, diễn tập nâng cấp và rollback
production	PC tại địa điểm; camera thật; profile `proxy` và `monitoring`	Vận hành; nâng cấp bằng bản phát hành có số hiệu

CHƯƠNG 4

Thuật toán analytics

Toàn bộ chương này là mã thuần Python, không phụ thuộc GPU hay mô hình. Đầu vào là danh sách detection có thời gian, đầu ra là số đếm, trạng thái và phiên. Nhờ vậy mọi thuật toán đều kiểm thử được bằng dữ liệu dựng sẵn và bằng detection đã lưu từ trước (mục 9.1).

4.1 Lọc detection

Mỗi detection $b = (x_1, y_1, x_2, y_2, s)$ được đưa về tọa độ ảnh gốc (bỏ letterbox) rồi đi qua các bộ lọc theo thứ tự:

1. **Lớp và độ tin cậy:** chỉ giữ lớp `person` với $s \geq s_{\text{min}}$. NMS chạy trong hậu xử lý, hoặc được tích hợp vào mô hình khi xuất nếu phiên bản Ultralytics đang dùng hỗ trợ.
2. **Kích thước:** bỏ hộp có chiều cao $h = y_2 - y_1 < h_{\text{min}}$ pixel. Ngưỡng này loại phần lớn nhiễu ở xa.
3. **Vùng loại trừ:** bỏ hộp có điểm neo nằm trong một vùng loại trừ (poster, màn hình chiếu, gương, cửa kính nhìn ra hành lang).

4.2 Gán người vào vùng

Điểm neo cấu hình được. Thay vì cố định điểm giữa cạnh dưới như brief v2.0, điểm neo được tham số hóa theo vị trí dọc $\alpha \in [0, 1]$:

$$p_\alpha(b) = \left(\frac{x_1 + x_2}{2},\ y_1 + \alpha\,(y_2 - y_1) \right). \tag{4.1}$$

$\alpha = 1$ là điểm ở chân (phù hợp người đứng ở sảnh), $\alpha = 0{,}5$ là tâm hộp (mặc định trong phòng, vì chân người ngồi thường bị bàn che).

Hai cách kiểm tra.

- **Theo điểm** (`assign: point`): b thuộc z nếu $p_\alpha(b) \in Z$. Phù hợp vùng nhỏ như từng chỗ ngồi.
- **Theo chồng lấn** (`assign: overlap`): b thuộc z nếu tỉ lệ $r_z(b) = |b \cap Z| / |b| \geq \tau$. Phù hợp vùng lớn như cả phòng, ổn định hơn khi hộp bị cắt ở mép.

Quy tắc nhóm. Vùng được khai báo theo nhóm g (ví dụ `seats` và `room`). Trong mỗi nhóm, một detection thuộc tối đa một vùng: nếu nhiều vùng cùng thỏa, chọn vùng có $r_z(b)$ lớn nhất, hòa thì chọn vùng có diện tích nhỏ hơn. Giữa các nhóm khác nhau, một người được đếm ở mỗi nhóm một lần, nên một người có thể vừa thuộc `room/lab` vừa thuộc `seats/lab-03`. Số đếm thô của vùng là

$$c_z(k) = \left| \{\, b \in \mathcal{D}_k : \text{vùng được gán của } b \text{ trong nhóm } g(z) \text{ là } z \,\} \right|, \tag{4.2}$$

với $\mathcal{D}_k$ là tập detection đã lọc của mẫu k. Kiểm tra cấu hình cảnh báo khi hai vùng cùng nhóm chồng lên nhau quá 10% diện tích.

> **Vẽ vùng theo chỗ điểm neo xuất hiện, không theo sàn nhà**
> Trên ảnh phối cảnh, tâm hộp của người ngồi nằm ở vùng ghế và phần thân dưới, thường cao hơn mặt sàn nơi đặt ghế. Công cụ vẽ vùng (mục 5.6) hiển thị điểm neo của detection hiện tại để vẽ đa giác bao đúng chỗ các điểm neo rơi vào.

4.3 Làm mượt theo thời gian

Detection của người ngồi yên thường chớp tắt: mất một vài mẫu khi người đó cúi xuống, quay lưng hoặc bị người khác đi ngang che. Bộ lọc trung vị trượt với cửa sổ lẻ W loại bỏ các mất mát ngắn này:

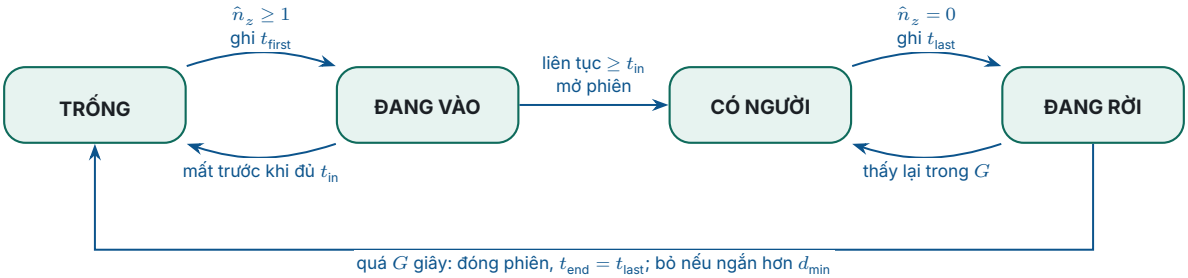
$$\hat{n}_z(k) = \text{median}\{\, c_z(k-W+1), \dots, c_z(k) \,\}. \tag{4.3}$$

Trung vị loại được nhiễu kéo dài dưới $(W+1)/2$ mẫu và giữ nguyên bước nhảy thật sau độ trễ $(W-1)/2$ mẫu. Trung bình trượt không làm được điều này: nó tạo giá trị lẻ và làm nhòe bước nhảy.

> **Mất dữ liệu không phải là "0 người"**
> Khi camera mất kết nối hoặc không có mẫu nào trong T_{stale} giây, trạng thái vùng là `UNKNOWN`, không phải 0. Các mẫu `UNKNOWN` bị loại khỏi K_T khi tính F_z và U_z, và mọi báo cáo kèm theo *độ phủ dữ liệu*: tỉ lệ thời gian mở cửa có dữ liệu hợp lệ. Nếu ghi 0 khi mất camera, báo cáo sẽ kết luận sai rằng phòng bị bỏ trống.

4.4 Trạng thái chỗ ngồi và phiên

Với vùng loại `seat` (thường $C_z = 1$), phiên được tính từ trạng thái của chỗ ngồi chứ không từ `track_id`. Cách này không bị ảnh hưởng khi tracker đổi ID, và vẫn chạy khi tắt tracking. Các ngưỡng đặt theo *giây*, không theo số khung, để không phụ thuộc f_s.



Hình 4.1: Máy trạng thái của một chỗ ngồi. Phiên bắt đầu tại lần đầu thấy người, không phải lúc đủ t_{in}.

Một phiên được ghi khi đóng, gồm `zone_id`, $t_{\text{start}} = t_{\text{first}}$, $t_{\text{end}} = t_{\text{last}}$, thời lượng $d = t_{\text{end}} - t_{\text{start}}$ và phương pháp (`seat_state`). Ba tham số chính là t_{in} (lọc người đi ngang qua), G (chịu được vắng mặt ngắn hoặc mất detection) và $d_{\min}$ (bỏ phiên quá ngắn). Khi engine khởi động lại, các phiên đang mở được đóng tại t_{last} với cờ `interrupted = true`, để báo cáo phân biệt được.

4.5 Phiên theo track trong vùng mở

Với vùng loại `area` (khu đứng chờ, quầy dịch vụ, cả phòng), thời gian lưu trú phải tính theo từng người, nên cần tracker.

Tracker ở tần số mẫu thấp. ByteTrack ghép detection theo IoU với dự đoán Kalman. Ở 2 FPS, người đi bộ dịch chuyển xa giữa hai mẫu nên dễ mất ghép, nhưng người ngồi hoặc đứng yên vẫn ghép tốt. Tham số `frame_rate` phải đặt bằng f_s, không bằng FPS nguồn. Trong cài đặt ByteTrack gốc, số khung một track mất dấu còn được giữ là $\lfloor \text{frame_rate}/30 \cdot \text{track_buffer} \rfloor$, nên muốn giữ G giây thì đặt track_buffer $\approx 30\,G$. Cần kiểm tra lại công thức này với thư viện và phiên bản đang dùng.

Nối track bị đứt. Khi tracker vẫn đổi ID, bước nối track chạy sau tracker:

1. Khi track a mất dấu bên trong vùng z tại t_a, ở vị trí p_a, với chiều cao hộp h_a: đưa a vào danh sách chờ của z trong G giây.
2. Khi track mới b xuất hiện lần đầu trong z tại $t_b \leq t_a + G$, ở vị trí p_b với $\|p_b - p_a\| \leq \beta\, h_a$: b là ứng viên nối với a.
3. Ghép tham lam theo khoảng cách nhỏ nhất, mỗi track chờ chỉ ghép một lần. Chuỗi track đã nối là một phiên.

4. Hết G mà không ai nối: đóng phiên tại lần cuối thấy.

Thời gian lưu trú của phiên là $d = t_{\text{last}} - t_{\text{first}}$ của cả chuỗi. Báo cáo kèm *tỉ lệ phân mảnh*: số phiên dự đoán chia cho số phiên thật được khớp. Thời gian lưu trú trong vùng mở là số liệu phụ, luôn báo cáo kèm giới hạn.

4.6 Đếm qua cửa (tùy chọn)

Tính năng bật bằng `door_counter.enabled`, dùng camera nhìn cửa ở $f_s = 10$ FPS. Logic giữ từ brief v2.0 và sửa lỗi đếm sót: với vạch AB và điểm neo P_t (ở cửa dùng $\alpha = 1$), khoảng cách có dấu là

$$s_t = \frac{(B - A) \times (P_t - A)}{\|B - A\|}. \tag{4.4}$$

Một lượt qua được ghi khi s đổi dấu sau khi đã ra khỏi dải trễ $|s| > \delta$ ở cả hai phía và giao điểm nằm trong đoạn AB. Chỉ đếm track đã tồn tại tối thiểu N_{min} mẫu. Một track được đếm lại cùng chiều *sau khi* đã qua theo chiều ngược, nên người vào, ra rồi vào lại được tính đúng 2 lượt vào. Thời gian chờ (cooldown) giữa hai lượt ngược chiều chống rung ở vạch.

Số người trong phòng tính từ cửa là $N(t) = \max(0,\ N(t_0) + \text{IN} - \text{OUT})$. Con số này trôi theo thời gian, nên được hiệu chỉnh: đặt về 0 lúc đóng cửa, và đặt về 0 khi camera trong phòng báo $\hat{n}_{\text{room}} = 0$ liên tục ít nhất 5 phút. Mỗi lần hiệu chỉnh ghi lại độ trôi $|N - \hat{n}_{\text{room}}|$ để đánh giá.

4.7 Tổng hợp theo thời gian

- **Mỗi giây:** writer cập nhật bảng `zone_live` (giá trị mới nhất, trạng thái, thời điểm) cho dashboard.
- **Mỗi 10 giây:** ghi `occupancy_10s` gồm trung bình và cực đại của $\hat{n}_z$, cùng tỉ lệ mẫu hợp lệ.
- **Mỗi phút:** ghi `occupancy_1m` gồm trung bình, cực đại, p95 của $\hat{n}_z$, tỉ lệ mẫu có người, tỉ lệ mẫu hợp lệ. p95 tính trong RAM từ các mẫu của phút đó vì không tính lại được từ giá trị trung bình.
- **Theo giờ, ngày, tuần:** tính bằng truy vấn từ `occupancy_1m` (trung bình có trọng số theo số mẫu hợp lệ).

Thời gian lưu dưới dạng UTC. Giờ mở cửa, ngày nghỉ và múi giờ của địa điểm (`Asia/Ho_Chi_Minh`) được áp dụng khi truy vấn, nên sửa giờ mở cửa không cần tính lại dữ liệu cũ.

4.8 Tham số mặc định

Tham số	Mặc định	Khoảng thử	Ý nghĩa
`sample_fps` f_s	2 (phòng), 10 (cửa)	1–5; 8–15	Tần số lấy mẫu theo camera
`imgsz`	640	480–1280	Kích thước đầu vào detector
`conf` s_{min}	0,30	0,20–0,45	Ngưỡng độ tin cậy
`min_box_h` h_{min}	24 px	16–48	Chiều cao hộp tối thiểu
`anchor_y` α	0,5	0,4–1,0	Vị trí dọc của điểm neo
`overlap_min` τ	0,3	0,2–0,5	Ngưỡng chồng lấn khi `assign: overlap`
`smooth_window` W	5 mẫu	3–9 (lẻ)	Cửa sổ trung vị
`stale_s` T_{stale}	10 s	5–30	Không có mẫu quá lâu thì `UNKNOWN`
`seat.t_in`	10 s	5–30	Thời gian liên tục để mở phiên
`seat.grace_s` G	60 s	30–180	Ân hạn trước khi đóng phiên
`seat.min_session_s` d_{min}	30 s	15–120	Bỏ phiên ngắn hơn
`stitch.max_gap_s`	30 s	10–90	Thời gian chờ nối track
`stitch.max_dist` β	0,5	0,3–1,0	Khoảng cách nối, theo bội số chiều cao hộp

Tham số	Mặc định	Khoảng thử	Ý nghĩa
`door.hysteresis_px` δ	10 px	5–25	Dải trễ quanh vạch
`door.min_track_age` $N_{\min}$	5 mẫu	3–10	Tuổi track tối thiểu
`door.cooldown_s`	2 s	1–5	Chờ giữa hai lượt ngược chiều

Mọi tham số chỉnh trên tập dev (mục 9.2), không bao giờ chỉnh trên tập test.

CHƯƠNG 5

Đặc tả chức năng

5.1 Yêu cầu chức năng và tiêu chí chấp nhận

Mã	Yêu cầu	Mô tả	Tiêu chí chấp nhận
FR01	Nguồn video thống nhất	Reader chỉ đọc RTSP từ MediaMTX. Camera IP khai báo làm `source` trong MediaMTX; webcam và MP4 được đẩy lên bằng FFmpeg.	Ba loại nguồn chạy cùng lúc ở staging; mã reader có một lớp nguồn duy nhất
FR02	Timeout và kết nối lại	Timeout mở 5 s, timeout đọc 5 s; backoff 0,5 → 1 → 2 → 4 → 5 s (tối đa 5 s); trạng thái `ONLINE`, `RECONNECTING`, `OFFLINE` (sau 60 s không kết nối được).	Đạt mọi kịch bản bơm lỗi mục 9.5; đạt M6
FR03	Lấy mẫu và bộ đệm	f_s theo camera; mỗi camera một ô shared memory; đếm số mẫu lấy, số mẫu bị ghi đè trước khi xử lý.	Mẫu bị bỏ $\leq 1\%$ ở cấu hình chuẩn; độ trễ không tăng dần khi nguồn nhanh hơn khả năng xử lý
FR04	Detector cấu hình được	Backend `ort-cuda`, `ort-cpu`, `torch`; mô hình, `imgsz`, FP16, kích thước batch ở cấp engine; `conf`, `min_box_h` ở cấp camera.	Đổi backend chỉ bằng cấu hình; backend CPU chạy được toàn bộ smoke test trong CI
FR05	Không gian, vùng, vùng loại trừ	Site → không gian (space) → vùng. Vùng có nhóm, loại (`seat`, `area`, `room`), sức chứa, cách gán, điểm neo. Mỗi vùng thuộc một camera; một không gian có thể gồm vùng của nhiều camera.	Cấu hình sai làm đúng camera đó báo lỗi rõ ràng, camera khác chạy tiếp; cảnh báo khi vùng cùng nhóm chồng lấn
FR06	Occupancy trực tiếp	`zone_live` cập nhật mỗi giây; trạng thái `UNKNOWN` khi quá T_{stale} không có mẫu.	Đạt M5; mất camera hiển thị `UNKNOWN`, không hiển thị 0
FR07	Phiên chỗ ngồi	Máy trạng thái mục 4.4.	Unit test các kịch bản đều xanh; đạt M3
FR08	Phiên vùng mở	Tracking theo camera và nối track mục 4.5; bật bằng `tracking.enabled`.	Báo cáo tỉ lệ phân mảnh trên tập test
FR09	Đếm qua cửa (tùy chọn)	Mục 4.6; bật bằng `door_counter.enabled`.	Unit test: đi qua, đứng trên vạch, quay đầu, vào–ra–vào, mất dấu ngắn
FR10	Tổng hợp và lưu giữ	Bảng 10 s, 1 phút, phiên; chính sách lưu giữ chạy hằng ngày.	Dữ liệu quá hạn bị xóa; dung lượng cơ sở dữ liệu tăng có giới hạn
FR11	REST API	Các endpoint mục 5.4; có phiên bản `/api/v1`; tài liệu OpenAPI tự sinh; phân trang bằng con trỏ.	Kiểm thử hợp đồng sinh từ OpenAPI đều xanh
FR12	WebSocket trực tiếp	Đẩy giá trị vùng khi thay đổi, tối đa mỗi giây một lần.	Dashboard cập nhật không cần tải lại trang
FR13	Dashboard	Các màn hình mục 5.5. Mọi thư viện JS phục vụ từ máy chủ, không tải từ CDN.	Chạy trong mạng không có Internet; tải trang dưới 2 s trong LAN
FR14	Công cụ vẽ vùng	Mục 5.6.	Không có ảnh nào được ghi ra đĩa (kiểm thử quyền riêng tư)

Mã	Yêu cầu	Mô tả	Tiêu chí chấp nhận
FR15	Xuất dữ liệu	CSV theo vùng hoặc không gian, theo khoảng; báo cáo tuần dạng HTML in được.	File CSV mở được bằng Excel, đúng múi giờ địa phương
FR16	Quan sát được	Log JSON có `camera_id`, `zone_id`; metric mục 6.3; `/health` (còn sống) và `/ready` (sẵn sàng).	Cảnh báo bật đúng trong các kịch bản bơm lỗi
FR17	Bảo mật	Mật khẩu quản trị băm; token API có hai phạm vi `read` và `admin`; không có mật khẩu mặc định.	Gọi endpoint quản trị không có token trả 401; script cài đặt tự sinh bí mật
FR18	Đóng gói và triển khai	Compose có profile; image gắn số phiên bản; script cài đặt, nâng cấp, rollback; gói cài offline.	Đạt M8
FR19	Sao lưu và phục hồi	Sao lưu trực tuyến hằng đêm, giữ 14 bản; script phục hồi.	Diễn tập phục hồi ở staging đạt trước mỗi bản phát hành
FR20	Quyền riêng tư	Chế độ production từ chối mọi tùy chọn ghi khung hình.	Đạt M9

5.2 Cấu hình

Cấu hình chia thành một file site và mỗi camera một file. Mã định danh vùng là duy nhất trong toàn site.

Mã nguồn 5.1: `configs/site.yaml`

```
schema_version: 1
site:
  id: lib-floor2
  name: "Thu vien tang 2"
  timezone: Asia/Ho_Chi_Minh
spaces:
  - {id: library, name: "Thu vien", capacity: 16}
  - {id: lab-a,   name: "Phong lab A", capacity: 6}
opening_hours:
  mon-fri: ["07:30-21:00"]
  sat:     ["08:00-17:00"]
  sun:     []
holidays: ["2027-01-01"]
engine:
  backend: ort-cuda           # ort-cuda | ort-cpu | torch
  model: /models/yolo11s-b8.onnx
  imgsz: 640
  fp16: true
  max_batch: 8
  batch_wait_ms: 50
retention:
  occupancy_10s_days: 30
  occupancy_1m_days: 400
  sessions_days: 400
privacy:
  debug_save_frames: false     # refused when OCC_ENV=production
```

Mã nguồn 5.2: `configs/cameras/lib-cam1.yaml`

```
schema_version: 1
camera_id: lib-cam1
source: rtsp://mediamtx:8554/lib-cam1
sample_fps: 2
detector: {conf: 0.30, min_box_h: 24}
exclusions:
```

```
  - {id: poster-1, polygon: [[1040,90],[1180,90],[1180,260],[1040,260]]}
zones:
  - id: lib-t1-s1
    space_id: library
    group: seats
    kind: seat
    capacity: 1
    assign: point
    anchor_y: 0.55
    polygon: [[212,402],[318,398],[322,488],[208,494]]
  - id: lib-room-east
    space_id: library
    group: room
    kind: room
    capacity: 8
    assign: overlap
    overlap_min: 0.3
    polygon: [[40,180],[1240,170],[1270,710],[20,715]]
seat: {t_in_s: 10, grace_s: 60, min_session_s: 30}
tracking: {enabled: false}
door_counter: {enabled: false}
```

Một không gian, nhiều camera

Thư viện lớn thường cần hai camera trở lên, và vùng nhìn của các camera có thể chồng nhau. Trong v1.0 không ghép người giữa các camera, nên **vùng** của các camera trong cùng không gian phải được vẽ không chồng lấn trên mặt bằng thật: mỗi camera phụ trách một phần. Số người của không gian là tổng các vùng nhóm `room` thuộc không gian đó. Kiểm tra cấu hình nhắc điều này khi một không gian có vùng từ nhiều camera.

5.3 Mô hình dữ liệu

Thời gian lưu dạng số nguyên mili giây UTC (`ts_ms`). Không có cột BLOB và không có `track_id`.

Mã nguồn 5.3: `src/occ/storage/schema.sql` (bản đầu, quản lý bằng Alembic)

```
CREATE TABLE camera_status (
  camera_id TEXT NOT NULL, ts_ms INTEGER NOT NULL,
  state TEXT NOT NULL,              -- ONLINE | RECONNECTING | OFFLINE | ERROR
  detail TEXT,
  PRIMARY KEY (camera_id, ts_ms)) WITHOUT ROWID;

CREATE TABLE zone_live (
  zone_id TEXT PRIMARY KEY, space_id TEXT NOT NULL, ts_ms INTEGER NOT NULL,
  n_smoothed INTEGER, state TEXT NOT NULL);     -- OK | UNKNOWN

CREATE TABLE occupancy_10s (
  zone_id TEXT NOT NULL, ts_ms INTEGER NOT NULL,
  n_mean REAL, n_max INTEGER, valid_frac REAL NOT NULL,
  PRIMARY KEY (zone_id, ts_ms)) WITHOUT ROWID;

CREATE TABLE occupancy_1m (
  zone_id TEXT NOT NULL, ts_ms INTEGER NOT NULL,
  n_mean REAL, n_max INTEGER, n_p95 REAL,
  occupied_frac REAL, valid_frac REAL NOT NULL, n_samples INTEGER NOT NULL,
  PRIMARY KEY (zone_id, ts_ms)) WITHOUT ROWID;

CREATE TABLE sessions (
  session_id TEXT PRIMARY KEY, zone_id TEXT NOT NULL,
  start_ms INTEGER NOT NULL, end_ms INTEGER NOT NULL, duration_s REAL NOT NULL,
  method TEXT NOT NULL,             -- seat_state | track_stitch
  n_fragments INTEGER NOT NULL DEFAULT 1,
  interrupted INTEGER NOT NULL DEFAULT 0);
CREATE INDEX idx_sessions_zone_start ON sessions(zone_id, start_ms);
```

```
CREATE TABLE door_events (
  event_id TEXT PRIMARY KEY, line_id TEXT NOT NULL,
  ts_ms INTEGER NOT NULL, direction TEXT NOT NULL);   -- IN | OUT
CREATE TABLE door_reconcile (
  line_id TEXT NOT NULL, ts_ms INTEGER NOT NULL, drift INTEGER NOT NULL,
  PRIMARY KEY (line_id, ts_ms)) WITHOUT ROWID;
```

Bảng	Lưu giữ	Dung lượng ước tính (50 vùng)
`zone_live`	chỉ giá trị hiện tại	không đáng kể
`occupancy_10s`	30 ngày	khoảng 20 MB/ngày, khoảng 650 MB tổng*
`occupancy_1m`	400 ngày	khoảng 4 MB/ngày, khoảng 1,7 GB tổng*
`sessions`	400 ngày	phụ thuộc lưu lượng; thường dưới 1 MB/ngày*
`camera_status`, `door_*`	90 ngày	nhỏ
Log container	xoay vòng 5 × 20 MB mỗi dịch vụ	tối đa khoảng 100 MB mỗi dịch vụ

5.4 REST API

Endpoint	Quyền	Mô tả
`GET /health`	không	Tiến trình còn sống; dùng cho healthcheck của Docker
`GET /ready`	không	Sẵn sàng: đọc được cơ sở dữ liệu, engine báo heartbeat trong 10 s gần nhất
`GET /metrics`	mạng nội bộ	Metric Prometheus
`GET /api/v1/spaces`	read	Không gian, sức chứa, số người hiện tại, số chỗ trống, trạng thái
`GET /api/v1/zones?space_id=`	read	Vùng, cấu hình rút gọn, giá trị trực tiếp
`GET /api/v1/cameras`	read	Trạng thái, f_s thực tế, tuổi khung cuối, tỉ lệ mẫu bị bỏ
`GET /api/v1/occupancy`	read	Chuỗi thời gian theo `zone_id` hoặc `space_id`, `from`, `to`, `interval` (10s, 1m, 1h, 1d)
`GET /api/v1/utilization`	read	F, U, P, P^{95}, H và độ phủ dữ liệu; `group_by` = `hour`, `day`, `weekday_hour`
`GET /api/v1/sessions`	read	Phiên theo vùng và khoảng thời gian; phân trang `limit`, `cursor`
`GET /api/v1/door/counts`	read	Lượt IN/OUT theo khoảng (khi bật đếm cửa)
`GET /api/v1/export.csv`	read	Xuất `kind` = `occupancy` hoặc `sessions`, theo giờ địa phương
`WS /api/v1/ws/live`	read	Đẩy thay đổi của `zone_live` và trạng thái camera
`GET /api/v1/public/availability`	công khai	Số chỗ trống theo không gian; tắt được; không có lịch sử
`GET /api/v1/admin/cameras/{id}/snapshot`	admin	Ảnh chụp đã làm mờ người, chỉ cho công cụ vẽ vùng
`POST /api/v1/admin/config/validate`	admin	Kiểm tra một cấu hình camera, trả danh sách lỗi và cảnh báo
`PUT /api/v1/admin/cameras/{id}/config`	admin	Lưu cấu hình đã kiểm tra, giữ bản cũ, yêu cầu engine nạp lại

Mã nguồn 5.4: Ví dụ phản hồi `GET /api/v1/utilization?space_id=library&from=2026-11-02&to=2026-11-08&group_by=day`

```
{
  "space_id": "library", "capacity": 16, "timezone": "Asia/Ho_Chi_Minh",
  "opening_hours_only": true,
  "items": [
    {"date": "2026-11-02", "fill_rate": 0.41, "time_utilization": 0.93,
     "peak": 15, "peak_p95": 13, "seat_hours": 88.6, "coverage": 0.998},
    {"date": "2026-11-03", "fill_rate": 0.47, "time_utilization": 0.95,
     "peak": 16, "peak_p95": 14, "seat_hours": 101.2, "coverage": 0.971}
  ]
}
```


5.5 Dashboard

Màn hình	Nội dung
Tổng quan trực tiếp	Sơ đồ mặt bằng (file SVG do quản trị viên tải lên, mỗi không gian gắn với một phần tử SVG) tô màu theo mức lấp đầy; nếu chưa có sơ đồ thì hiển thị dạng thẻ. Huy hiệu trạng thái camera.
Chi tiết không gian	Đường occupancy hôm nay so với mức điển hình (trung bình cùng thứ trong 4 tuần trước); thẻ F, U, cao điểm; phân bố thời lượng phiên.
Bản đồ nhiệt	Mức lấp đầy theo thứ trong tuần × giờ cho khoảng thời gian đã chọn.
So sánh không gian	Bảng sắp xếp theo F, U, P^{95}, kèm độ phủ dữ liệu; đánh dấu “thường quá tải” ($P^{95} \geq 0{,}9\,C$) và “dùng ít” (U thấp hoặc F thấp khi U cao).
Sức khỏe hệ thống	Trạng thái từng camera, f_s thực tế, mẫu bị bỏ, GPU, dung lượng đĩa, thời điểm sao lưu gần nhất, phiên bản đang chạy.
Trang chỗ trống công khai	Số chỗ trống theo không gian, tự làm mới; không có lịch sử, không cần đăng nhập.
Quản trị	Công cụ vẽ vùng, quản lý token, xem lịch sử thay đổi cấu hình.

5.6 Công cụ vẽ vùng

Vẽ vùng là bước lâu nhất khi lắp đặt ở địa điểm mới, nên cần một công cụ thay vì sửa tọa độ bằng tay.

1. Quản trị viên chọn camera. API gọi endpoint nội bộ của engine để lấy ảnh chụp.
2. Engine lấy khung mới nhất, chạy detector, làm mờ (pixelate) mọi hộp người cộng thêm lề 10%, mã hóa JPEG trong bộ nhớ và trả về. Kèm theo là tọa độ điểm neo p_α của các detection.
3. API giữ ảnh trong RAM tối đa 10 phút để hiển thị, không ghi ra đĩa, và ghi log kiểm toán “ai xem ảnh chụp lúc nào”.
4. Trình duyệt hiển thị ảnh, điểm neo hiện tại và các vùng đã có. Quản trị viên vẽ hoặc sửa đa giác, đặt nhóm, loại, sức chứa.
5. Khi lưu: API kiểm tra cấu hình, giữ bản cũ trong `configs/.history/`, ghi file mới, yêu cầu engine nạp lại camera đó.

CHƯƠNG 6

Yêu cầu phi chức năng

6.1 Cấu hình chuẩn để so sánh

Mọi số đo hiệu năng báo cáo ở cấu hình chuẩn sau, trừ khi ghi khác: 8 luồng 1280×720 H.264, nguồn 10 FPS (luồng phụ của camera hoặc MP4 mô phỏng), $f_s = 2$ FPS, mô hình cỡ s, `imgsz` 640, ONNX Runtime CUDA FP16, `max_batch` 8. Mỗi báo cáo phải ghi GPU, VRAM, CPU, RAM, phiên bản driver, CUDA, cuDNN, ONNX Runtime và commit hash.

> **Luồng mô phỏng không bằng camera thật**
> Luồng RTSP phát từ MP4 không có trễ mạng, mất gói hay lệch đồng hồ như camera thật. Benchmark trên luồng mô phỏng dùng để so sánh cấu hình; con số cuối cùng về số camera trên một GPU phải được xác nhận lại với ít nhất 2 camera thật ở staging.

6.2 Bảng yêu cầu

Nhóm	Yêu cầu	Cách kiểm
Hiệu năng	≥ 8 camera ở $f_s = 2$ trên một GPU, mẫu bị bỏ $\leq 1\%$ (M4)	Benchmark mục 9.4
Độ tươi	p95 của $t_5 - t_0 \leq 3$ s (M5)	Metric `occ_freshness_seconds`
Phục hồi	Camera về `ONLINE` ≤ 10 s sau khi nguồn sống lại; camera khác không bị ảnh hưởng (M6)	Kịch bản bơm lỗi
Ổn định	Soak 24 h: 0 crash, RAM tăng $< 10\%$ so với sau giờ đầu (M7); pilot 7 ngày, thời gian hoạt động $\geq 99\%$	Soak test, pilot
Tài nguyên	Ghi nhận GPU, VRAM, CPU, RAM theo số camera; chưa đặt ngưỡng	`nvidia-smi`, metric tiến trình
Lưu trữ	Dung lượng cơ sở dữ liệu có giới hạn nhờ chính sách lưu giữ; cảnh báo khi ổ đĩa đầy trên 80%	Kiểm thử lưu giữ; cảnh báo
Bảo mật	Không có bí mật trong Git; không có mật khẩu mặc định; container chạy bằng user không phải root; endpoint quản trị cần token `admin`	Quét bí mật trong CI; kiểm thử API
Quyền riêng tư	0 file ảnh hoặc video được ghi ở production (M9); không lưu `track_id`	Kiểm thử quyền riêng tư mục 9.7
Triển khai	Cài mới ≤ 15 phút bằng 1 lệnh; nâng cấp và rollback ≤ 5 phút, không mất dữ liệu (M8)	Diễn tập ở staging mỗi bản phát hành
Bảo trì	Kiểu dữ liệu kiểm tra bằng mypy; độ phủ test của gói `analytics` $\geq 90\%$; mỗi quyết định kiến trúc có một ADR	CI
Tái lập	Mọi bảng số đo sinh lại được bằng một lệnh từ commit và dữ liệu đã ghi	`make eval`, `make bench`
Tính di động	Ubuntu 22.04 hoặc 24.04 LTS là nền tảng chính; Windows 11 với WSL2 dùng cho dev; chế độ chỉ CPU cho 1–2 camera	Chạy CI trên Linux; thử thủ công trên WSL2

6.3 Metric và log

Metric	Kiểu	Ý nghĩa
`occ_camera_state{camera}`	gauge	0 OFFLINE, 1 RECONNECTING, 2 ONLINE, 3 ERROR
`occ_samples_total{camera}`	counter	Số mẫu đã lấy
`occ_samples_dropped_total{camera}`	counter	Mẫu bị ghi đè trước khi được xử lý
`occ_reconnects_total{camera}`	counter	Số lần kết nối lại
`occ_batch_size`	histogram	Kích thước batch thực tế
`occ_inference_seconds`	histogram	$t_2 - t_1$
`occ_freshness_seconds`	histogram	$t_5 - t_0$, đo ở API khi đẩy WebSocket
`occ_zone_occupancy{zone}`	gauge	$\hat{n}_z$ hiện tại; -1 khi `UNKNOWN`
`occ_db_write_seconds`, `occ_db_write_errors_total`	histogram, counter	Thời gian commit và số lỗi ghi
`occ_gpu_util`, `occ_gpu_mem_bytes`	gauge	Đọc bằng NVML
`occ_build_info{version,commit}`	gauge	Phiên bản đang chạy, luôn bằng 1

Log ghi dạng JSON ra stdout, mỗi dòng có `ts`, `level`, `service`, `version`, và `camera_id` hoặc `zone_id` khi liên quan. Docker xoay vòng log theo kích thước. Log không chứa URL camera có mật khẩu: mật khẩu trong URL RTSP được che trước khi ghi.

CHƯƠNG 7

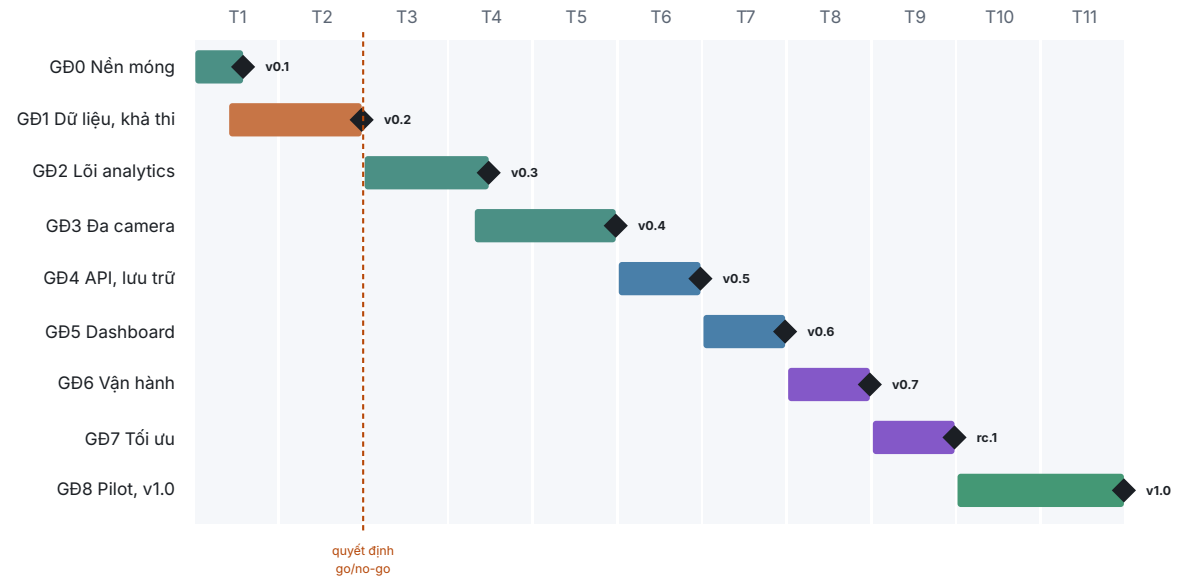
Lộ trình từ A đến Z

7.1 Nguyên tắc

1. **Bộ khung chạy được trước, tính năng sau.** Giai đoạn 0 dựng toàn bộ đường đi từ commit đến máy chạy thật (CI, image, Compose, script deploy) với các thành phần còn rỗng. Từ đó trở đi, mỗi thay đổi chỉ là thêm vào một hệ thống vốn đã deploy được.
2. **Nhánh `main` luôn deploy được.** Chỉ merge khi CI xanh. Tính năng chưa xong nằm sau cờ cấu hình, mặc định tắt.
3. **Mỗi giai đoạn kết thúc bằng một bản phát hành có số hiệu** và được cài lên staging bằng đúng script dùng cho production.
4. **Đo trước, tối ưu sau.** Bộ dữ liệu đánh giá và lệnh `make eval` có trước thuật toán analytics; benchmark có trước tối ưu.
5. **Rủi ro lớn làm trước.** Docker thấy GPU (ngày đầu), detector thấy người ngồi (tuần đầu), quyền riêng tư (từ thiết kế).

7.2 Tổng quan tiến độ

Ước tính cho một người làm khoảng 20–25 giờ mỗi tuần*. Các giai đoạn gối nhau một phần.



Hình 7.1: Tiến độ dự kiến theo tuần. Hình thoi là bản phát hành cuối mỗi giai đoạn.

Giai đoạn 0 · Nền móng và bộ khung chạy được — Tuần 1 (3 ngày) · v0.1.0

Mục tiêu. Có đường đi hoàn chỉnh từ commit đến hệ thống chạy trên máy staging, trước khi có tính năng nào.
Công việc.

1. **Ngày đầu tiên:** cài driver NVIDIA, Docker, NVIDIA Container Toolkit trên PC GPU và chạy `nvidia-smi` bên trong một container. Đây là rủi ro có thể chặn cả dự án, nên làm trước mọi việc khác.
2. Tạo repo theo Phụ lục A: `pyproject.toml` quản lý bằng uv; ruff, mypy, pytest, pre-commit; README khung, CHANGELOG, LICENSE; ADR đầu tiên trong `docs/adr/`.

3. Gói cấu hình: model Pydantic cho site và camera, `schema_version: 1`, lệnh `occ config validate`, cấu hình mẫu.
4. Engine rỗng: supervisor, một reader đọc RTSP và đếm khung, detector giả luôn trả 0 người, writer ghi `zone_live`, endpoint `/health` và `/metrics`.
5. API rỗng: `/health`, `/ready`, `/api/v1/spaces` đọc `zone_live`; một trang dashboard tối thiểu.
6. Compose: `mediamtx`, `sim-publisher` (profile `sim`), `engine`, `api`, có healthcheck; file `.env.example`.
7. CI: lint, type check, test, build image, smoke test toàn hệ thống bằng profile `sim`; đẩy image gắn commit hash lên GHCR.
8. Bản đầu của `deploy/install.sh`, `deploy/deploy.sh`, `deploy/rollback.sh`; cài thử lên staging.

Xong khi.

☐ Trên laptop không GPU, `git clone` rồi `make up-dev` chạy được toàn hệ thống.
☐ Staging chạy bằng `deploy.sh --version 0.1.0`; container engine thấy GPU.
☐ CI xanh trên `main`; image v0.1.0 có trên registry.

Bản deploy cuối giai đoạn. Hệ thống chạy end-to-end, dashboard hiển thị camera `ONLINE`; số người luôn bằng 0.

Giai đoạn 1 · Dữ liệu, baseline và phép thử khả thi — Tuần 1–2 · v0.2.0

Mục tiêu. Trả lời câu hỏi rủi ro nhất (detector có thấy người ngồi không) và có bộ dữ liệu đánh giá trước khi viết analytics.

Công việc.

1. Xin phép và quay clip ở ít nhất 2 không gian, tổng tối thiểu 6 giờ*, trải đều các mức đông (vắng, vừa, đông), ánh sáng (ban ngày, đèn, chập tối) và có đoạn phòng trống hoàn toàn để bắt false positive cố định. Ghi lại vị trí, độ cao, góc camera.
2. Chạy phép thử khả thi mục 2.1; ghi quyết định vào một ADR.
3. Công cụ `occ tools sample-frames` trích khung mỗi 15 s; gán nhãn theo giao thức mục 9.2; chia dev/test theo *ngày quay*, không chia theo khung, để tránh hai khung gần giống nhau rơi vào hai tập.
4. Đưa detector thật vào engine: xuất ONNX với batch động, chạy bằng ONNX Runtime CUDA và CPU.
5. Công cụ `occ tools cache-detections` lưu detection của mọi clip ra JSONL, để test và đánh giá analytics không cần GPU.
6. Đo baseline: recall và precision theo tư thế và mức che, thời gian suy luận theo kích thước batch.
7. Chốt các ngưỡng M1–M9 từ số đo baseline, ghi vào `docs/targets.md`.

Xong khi.

☐ Quyết định go/no-go được ghi lại kèm số đo.
☐ Tập test được đóng băng: danh sách file và mã băm nằm trong repo, không ai chỉnh tham số trên tập này.
☐ Dashboard hiển thị tổng số người theo từng camera.

Bản deploy cuối giai đoạn. Đếm tổng số người mỗi camera theo thời gian thực, chưa có vùng.

Giai đoạn 2 · Lõi analytics — Tuần 3–4 · v0.3.0

Mục tiêu. Toàn bộ thuật toán Chương 4, có test và có số đo độ chính xác trên tập test.

Công việc.

1. Giao diện `Clock` với `RealClock` và `FrameClock`.
2. Lọc detection, vùng loại trừ, gán vùng theo điểm và theo chồng lấn, quy tắc nhóm.
3. Làm mượt trung vị, trạng thái `UNKNOWN` khi thiếu mẫu, độ phủ dữ liệu.
4. Máy trạng thái chỗ ngồi và phiên, kể cả cờ `interrupted`.
5. Tracking theo camera và nối track (sau cờ `tracking.enabled`).
6. Đếm qua cửa (sau cờ `door_counter.enabled`; là mục cắt đầu tiên nếu chậm).
7. Writer tổng hợp 10 s và 1 phút; truy vấn chỉ số sử dụng theo giờ mở cửa và múi giờ.
8. Unit test và property test (mục 9.1); lệnh `make eval` chạy trên detection đã lưu và sinh bảng MAE, sai số F, U, chỉ số phiên.
9. Kiểm thử hồi quy độ chính xác trong CI trên một clip nhỏ có sẵn detection: CI đỏ nếu MAE xấu hơn baseline đã commit.

10. Chỉnh tham số trên tập dev; báo cáo kết quả trên tập test.

Xong khi.
☐ Test xanh; độ phủ test gói `analytics` $\geq 90\%$.
☐ `make eval` sinh lại đúng bảng kết quả từ commit hiện tại.
☐ Có số đo M1–M3 lần đầu trên tập test.
Bản deploy cuối giai đoạn. Occupancy và phiên theo vùng thật, nhưng vùng còn phải vẽ bằng tay trong YAML.

Giai đoạn 3 · Pipeline đa camera bền vững — Tuần 4–5 · v0.4.0

Mục tiêu. Chạy nhiều camera, tự phục hồi, không rò bộ nhớ, một camera lỗi không ảnh hưởng camera khác.
Công việc.

1. Reader cho từng camera: RTSP qua TCP, timeout mở và đọc, backoff tối đa 5 s, ba trạng thái, che mật khẩu trong log.
2. Ô shared memory cho từng camera; bộ đếm mẫu bị bỏ.
3. Engine suy luận gộp batch theo `max_batch` và `batch_wait_ms`; tự chuyển sang CPU khi cấu hình yêu cầu.
4. Supervisor: khởi động lại tiến trình con với backoff, heartbeat, đóng phiên đang mở khi lõi khởi động lại.
5. Nạp lại cấu hình theo từng camera; lỗi cấu hình chỉ dừng đúng camera đó.
6. Writer: một luồng ghi, gom giao dịch, chạy chính sách lưu giữ hằng ngày.
7. Script đẩy webcam (Linux dùng `v4l2`, Windows dùng `dshow`) và publisher mô phỏng cho N clip.
8. Bộ bơm lỗi `make chaos` cho mọi kịch bản mục 9.5.

Xong khi.
☐ Mọi kịch bản bơm lỗi đạt.
☐ 8 camera mô phỏng chạy trên staging, mẫu bị bỏ $\leq 1\%$.
☐ Soak sơ bộ 8 giờ không crash.
Bản deploy cuối giai đoạn. Nhiều camera, tự phục hồi; dùng được cho một phòng thật nếu chấp nhận vẽ vùng bằng tay.

Giai đoạn 4 · API và lưu trữ — Tuần 6 · v0.5.0

Mục tiêu. API đầy đủ, ổn định, có phiên bản, để dashboard và hệ thống khác dùng được.
Công việc.

1. Migration đầu tiên bằng Alembic; test trong CI nâng cấp từ cơ sở dữ liệu mẫu của bản phát hành trước.
2. Các endpoint mục 5.4 với model phản hồi Pydantic, OpenAPI, phân trang bằng con trỏ.
3. Xác thực: token phạm vi `read` và `admin`; mật khẩu quản trị băm bằng Argon2 hoặc bcrypt; CORS giới hạn.
4. WebSocket trạng thái trực tiếp.
5. Xuất CSV theo giờ địa phương.
6. Kiểm thử hợp đồng sinh từ OpenAPI (ví dụ Schemathesis) và kiểm thử tích hợp với cơ sở dữ liệu có dữ liệu mẫu.

Xong khi.
☐ Kiểm thử hợp đồng xanh; gọi endpoint quản trị không có token trả 401.
☐ Nâng cấp cơ sở dữ liệu từ v0.4 lên v0.5 không mất dữ liệu.
Bản deploy cuối giai đoạn. API dùng được cho bên thứ ba; dashboard vẫn ở mức tối thiểu.

Giai đoạn 5 · Dashboard và công cụ vẽ vùng — Tuần 7 · v0.6.0

Mục tiêu. Người không biết code tự lắp đặt camera, vẽ vùng và đọc báo cáo được.
Công việc.

1. Màn hình tổng quan trực tiếp, chi tiết không gian, bản đồ nhiệt, so sánh không gian, sức khỏe hệ thống (mục 5.5).
2. Trang chỗ trống công khai.
3. Công cụ vẽ vùng với ảnh đã làm mờ, lớp điểm neo, kiểm tra, lưu, nạp lại, lịch sử (mục 5.6).
4. Đưa mọi thư viện JS vào thư mục tĩnh, không dùng CDN.
5. Hai đến ba kiểm thử trình duyệt (ví dụ Playwright) cho các luồng chính: xem tổng quan, vẽ và lưu một vùng.

Xong khi.
☐ Một người mới, chỉ đọc tài liệu, vẽ xong vùng cho một camera trong ≤ 15 phút.
☐ Kiểm thử quyền riêng tư vẫn đạt sau khi có công cụ chụp ảnh.
Bản deploy cuối giai đoạn. Phần mềm đầy đủ tính năng người dùng; chưa đủ công cụ vận hành.

Giai đoạn 6 · Vận hành, bảo mật, sao lưu — Tuần 8 · v0.7.0

Mục tiêu. Người khác vận hành được hệ thống mà không cần hỏi tác giả.
Công việc.

1. Profile `monitoring`: Prometheus, Grafana, dashboard dạng JSON trong repo, luật cảnh báo mục 10.5.
2. Dịch vụ sao lưu hằng đêm bằng online backup của SQLite; script phục hồi; diễn tập phục hồi.
3. Hoàn thiện `install.sh`: kiểm tra driver, Docker, NVIDIA Container Toolkit; tự sinh bí mật; kéo image; chạy migration; chờ `/ready`. Thêm gói cài offline (image xuất bằng `docker save`) cho nơi không có Internet.
4. Nâng cấp và rollback: tự sao lưu trước migration; diễn tập ở staging.
5. Bảo mật: container chạy user không phải root; quét bí mật (ví dụ gitleaks) và quét image (ví dụ Trivy) trong CI; profile `proxy` với Caddy và HTTPS nội bộ.
6. Soak 24 giờ trên staging; sửa rò bộ nhớ nếu có.
7. Tài liệu `docs/deployment.md`, `docs/operations.md` (runbook), `docs/privacy.md`.

Xong khi.
☐ Đạt M7 và M8; diễn tập phục hồi đạt.
☐ Mọi cảnh báo đều có một mục trong runbook.
Bản deploy cuối giai đoạn. Phần mềm vận hành được bởi người khác.

Giai đoạn 7 · Tối ưu và benchmark — Tuần 9 · v0.9.0-rc.1

Mục tiêu. Biết chính xác hệ thống chạy được bao nhiêu camera, ở cấu hình nào, với độ chính xác bao nhiêu. Đóng băng tính năng.
Công việc.

1. Chạy ma trận benchmark mục 9.4 bằng `make bench`; lưu CSV kèm phần cứng và commit hash.
2. Kiểm tra độ chính xác sau FP16: MAE không xấu hơn FP32 quá 0,05 người/mẫu; F và U không lệch quá 1 điểm phần trăm.
3. Tinh chỉnh `max_batch`, `batch_wait_ms`, `imgsz`; thử decode bằng phần cứng và TensorRT EP nếu còn thời gian.
4. Xác định số camera tối đa trên GPU staging (M4).
5. Đóng băng tính năng; phát hành bản ứng viên và soak 24 giờ.

Xong khi.
☐ Báo cáo benchmark sinh lại được bằng một lệnh.
☐ Bản ứng viên qua soak 24 giờ.
Bản deploy cuối giai đoạn. Bản ứng viên phát hành, sẵn sàng cho pilot.

Giai đoạn 8 · Pilot và phát hành v1.0 — Tuần 10–11 · v1.0.0

Mục tiêu. Chứng minh phần mềm chạy được ở một địa điểm thật trong một tuần, theo đúng tài liệu triển khai.
Công việc.

1. Chuẩn bị địa điểm: biển thông báo, đồng ý của người phụ trách, lắp camera, mạng (Chương 11 và mục 10.3).
2. Cài bản ứng viên bằng `install.sh`, làm đúng từng bước trong `docs/deployment.md`; ghi mọi chỗ vướng và sửa tài liệu.
3. Nghiệm thu tại hiện trường (mục 9.8).
4. Chạy 7 ngày; theo dõi cảnh báo; kiểm tra nhanh mỗi ngày.
5. Báo cáo đánh giá cuối: độ chính xác trên tập test và trên số đếm tay ở pilot, số đo hệ thống, giới hạn đã biết.
6. Phát hành v1.0.0: CHANGELOG, tag, GitHub Release kèm gói triển khai; video demo 1–2 phút đã làm mờ người; README hoàn chỉnh.

Xong khi. Mọi mục trong Definition of Done (Chương 13) đều đạt.
Bản deploy cuối giai đoạn. v1.0.0.

7.3 Dự phòng và phiên bản rút gọn

Thứ tự cắt khi chậm tiến độ

Cắt theo thứ tự: (1) đếm qua cửa, (2) phiên trong vùng mờ và nối track, (3) Grafana (giữ endpoint `/metrics`), (4) sơ đồ mặt bằng SVG (giữ dạng thẻ), (5) TensorRT. **Không bao giờ cắt:** phép thử khả thi, tập test đóng băng, kiểm thử bơm lỗi, kiểm thử quyền riêng tư, sao lưu, rollback.

Nếu chỉ có 5–6 tuần cho portfolio

Làm đầy đủ Giai đoạn 0–3; Giai đoạn 4 chỉ gồm `spaces`, `occupancy`, `utilization`; Giai đoạn 5 chỉ gồm màn hình tổng quan và bản đồ nhiệt, vẽ vùng bằng một script nhỏ; soak 8 giờ thay vì 24 giờ. Kết quả là v0.6 vẫn deploy được bằng một lệnh và đủ để demo. README cần ghi rõ phần nào chưa làm.

CHƯƠNG 8

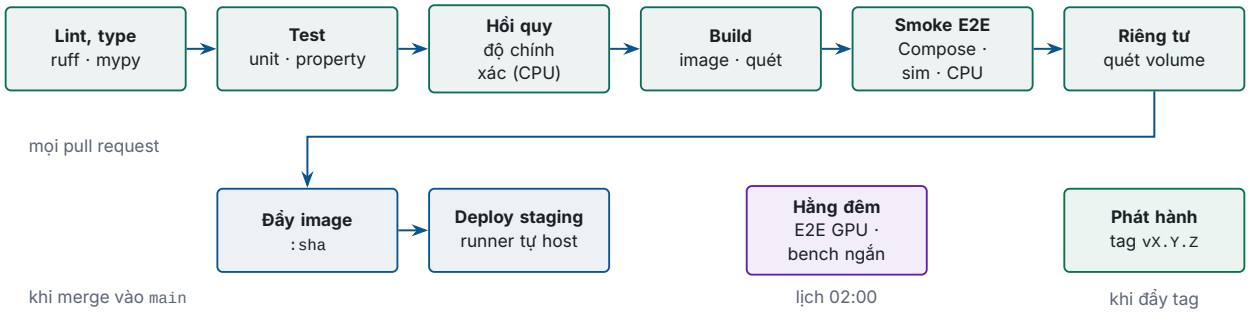
Quy trình phát triển và phát hành liên tục

"Deploy được bất cứ lúc nào" không phải là một tính năng làm ở cuối dự án, mà là kết quả của một quy trình áp dụng từ commit đầu tiên. Chương này mô tả quy trình đó.

8.1 Nhánh và commit

- **Trunk-based:** nhánh tính năng sống tối đa 2–3 ngày, merge vào `main` qua pull request khi CI xanh. Không có nhánh `develop`.
- **Conventional Commits** (`feat:`, `fix:`, `perf:`, `docs:`, `chore:`; thêm `!` khi thay đổi không tương thích) để sinh CHANGELOG tự động.
- **Mẫu pull request** có danh sách kiểm: đã thêm test, đã cập nhật tài liệu, có migration hay không, có thay đổi cấu hình hay không.
- **Tính năng chưa xong** nằm sau cờ cấu hình mặc định tắt, nên merge sớm mà không ảnh hưởng bản đang chạy.

8.2 Pipeline CI/CD



Hình 8.1: Các bước CI/CD. Hàng trên chặn merge; hàng dưới chạy sau khi merge, theo lịch hoặc khi gắn tag.

Bước	Nội dung	Khi chạy
Lint, type	ruff (gồm luật cấm `cv2.imwrite` và `VideoWriter` ngoài `occ/tools/`), mypy	Mọi PR, chặn merge
Test	Unit test, property test; báo cáo độ phủ	Mọi PR, chặn merge
Hồi quy độ chính xác	`make eval` trên clip fixture có sẵn detection; so với baseline đã commit	Mọi PR, chặn merge
Build	Build image bằng buildx có cache; quét bí mật và lỗ hổng image	Mọi PR, chặn merge
Smoke E2E	Compose profile `sim`, backend `ort-cpu`, mô hình cỡ n, 2 camera, 3 phút: `/ready`, `zone_live` thay đổi, kiểm thử hợp đồng API	Mọi PR, chặn merge
Riêng tư	Quét volume và hệ thống file container tìm file ảnh hoặc video; kiểm tra schema không có BLOB; log không có mật khẩu	Mọi PR, chặn merge
Đẩy image	Image gắn `:sha` lên GHCR	Merge vào `main`
Deploy staging	Runner tự host trên PC staging chạy `deploy.sh --version sha-...`	Merge vào `main` (tùy chọn)
Hằng đêm	E2E trên GPU, benchmark ngắn, soak 2 giờ	Theo lịch

Bước	Nội dung	Khi chạy
Phát hành	Image `:X.Y.Z`, GitHub Release kèm gói triển khai và CHANGELOG	Khi đẩy tag `vX.Y.Z`

> **CI không cần GPU**
> Nhờ backend `ort-cpu` và detection đã lưu sẵn, toàn bộ các bước chặn merge chạy trên runner miễn phí. GPU chỉ cần cho bước hằng đêm và benchmark.

8.3 Phiên bản và tương thích

- **Ứng dụng:** SemVer. MAJOR khi thay đổi không tương thích ở API hoặc schema cấu hình; MINOR khi thêm tính năng; PATCH khi sửa lỗi. Mọi image cùng một bản phát hành có cùng số phiên bản.
- **API:** `/api/v1` giữ ổn định. Thay đổi phá vỡ đi vào `/api/v2`, và `/api/v1` được giữ thêm ít nhất một bản MINOR.
- **Cấu hình:** trường `schema_version`; lệnh `occ config migrate` nâng cấu hình cũ; engine từ chối cấu hình mới hơn phiên bản nó hiểu.
- **Cơ sở dữ liệu:** Alembic, chỉ đi tiến. Migration theo kiểu *mở rộng rồi thu gọn*: bản N thêm cột hoặc bảng mới nhưng vẫn ghi dữ liệu theo cách cũ; bản N+1 mới bỏ phần cũ. Nhờ vậy mã của bản N−1 vẫn chạy trên schema của bản N, và rollback thường không cần phục hồi dữ liệu. Migration nào không tuân theo quy tắc này phải ghi trong CHANGELOG là "rollback cần phục hồi bản sao lưu".

8.4 Cờ tính năng

Cờ	Mặc định	Bật khi
`tracking.enabled`	tắt	Camera có vùng `area` cần thời gian lưu trú theo người
`door_counter.enabled`	tắt	Có camera nhìn cửa ở 10 FPS
`api.public_availability`	tắt	Tổ chức đồng ý công khai số chỗ trống
`dashboard.floorplan`	tắt	Đã tải lên sơ đồ mặt bằng SVG
`privacy.debug_save_frames`	tắt	Chỉ ở dev; bị từ chối khi `OCC_ENV=production`

8.5 Danh sách kiểm trước khi phát hành

- ☐ CI xanh trên commit được gắn tag; hồi quy độ chính xác không xấu đi.
- ☐ Soak 24 giờ trên staging với bản ứng viên (từ v0.7 trở đi).
- ☐ Diễn tập nâng cấp từ bản trước và rollback về bản trước trên staging.
- ☐ Diễn tập phục hồi bản sao lưu.
- ☐ CHANGELOG ghi rõ migration, thay đổi cấu hình, thay đổi API, và rollback có cần phục hồi hay không.
- ☐ Tài liệu triển khai và runbook khớp với bản này.

8.6 Cài đặt, nâng cấp, rollback

Máy đích chỉ cần driver NVIDIA, Docker Engine với Compose plugin và NVIDIA Container Toolkit. Mọi thứ khác nằm trong gói triển khai.

Mã nguồn 8.1: Ba lệnh vận hành chính

```
# Fresh install
tar xzf occ-deploy-1.0.0.tar.gz && cd occ-deploy-1.0.0
sudo ./install.sh                # check env, generate secrets, pull images, migrate, wait /ready
```

```
# Upgrade
./deploy.sh --version 1.1.0       # backup -> pull -> migrate -> restart -> wait /ready -> auto-rollback on failure

# Roll back to the previous version
./rollback.sh                     # previous version; restores backup only if CHANGELOG requires it
```

`deploy.sh` làm gì.

1. Kiểm tra phiên bản đích tồn tại trên registry (hoặc trong gói offline) và đọc ghi chú tương thích.
2. Sao lưu cơ sở dữ liệu và thư mục cấu hình, ghi phiên bản hiện tại vào `deploy/.previous`.
3. Kéo image mới; chạy `occ config migrate --check`; chạy migration cơ sở dữ liệu.
4. Khởi động lại dịch vụ với `OCC_VERSION` mới; chờ `/ready` tối đa 120 s.
5. Nếu không sẵn sàng: tự rollback (bước 2 đảo ngược), ghi log lý do, thoát với mã lỗi.

CHƯƠNG 9

Kiểm thử và đánh giá

9.1 Các tầng kiểm thử

Tầng	Phạm vi	Công cụ	Khi chạy
Unit	Hình học, gán vùng, làm mượt, máy trạng thái, nối track, đếm cửa, tổng hợp, kiểm tra cấu hình	pytest	Mọi PR
Property	Bất biến đúng với mọi đầu vào (xem bên dưới)	Hypothesis	Mọi PR
Hồi quy độ chính xác	Analytics trên detection đã lưu của clip fixture	`make eval`	Mọi PR
Tích hợp	Reader với MediaMTX thật; writer với SQLite thật; API với cơ sở dữ liệu có dữ liệu mẫu	pytest, Docker	Mọi PR
Hợp đồng API	Mọi endpoint đúng với OpenAPI	Schemathesis	Mọi PR
Smoke E2E	Toàn hệ thống bằng Compose, backend CPU	pytest, Compose	Mọi PR
Trình duyệt	Xem tổng quan, vẽ và lưu vùng	Playwright	Mọi PR (từ v0.6)
Hiệu năng	Ma trận benchmark	`make bench`	Hằng đêm (ngắn), trước phát hành (đủ)
Bơm lỗi	Kịch bản mục 9.5	`make chaos`	Trước phát hành
Soak	24 giờ liên tục	`make soak`	Trước phát hành (từ v0.7)
Quyền riêng tư	Mục 9.7	pytest, script quét	Mọi PR

Kịch bản unit test bắt buộc.

- **Chỗ ngồi:** ngồi liên tục; mất 1–2 mẫu; rời 30 s rồi quay lại (vẫn một phiên); rời 90 s (thành hai phiên); đi ngang 5 s (không có phiên); camera mất kết nối giữa phiên (`UNKNOWN`; quá G thì đóng phiên tại t_{last} với cờ `interrupted`).
- **Gán vùng:** hộp ở biên giữa hai vùng; hai vùng cùng nhóm chồng lấn; điểm neo trong vùng loại trừ; vùng nhiều nhóm.
- **Nối track:** đổi ID tại chỗ (phải nối); người khác xuất hiện gần đó sau khi người đầu rời đi quá G (không được nối).
- **Đếm cửa:** đi qua; đứng trên vạch; quay đầu giữa chừng; vào, ra, vào lại (2 IN); mất dấu ngắn khi đang qua vạch.
- **Tổng hợp:** khoảng có mẫu `UNKNOWN`; ranh giới ngày theo giờ địa phương; ngày nghỉ; đổi giờ mở cửa.

Bất biến cho property test. $\hat{n}_z(k) \geq 0$; $\hat{n}_z(k)$ không vượt quá số detection lớn nhất trong cửa sổ; các phiên của cùng một chỗ ngồi không chồng nhau; $H_z(T) \leq C_z \cdot |T|$ khi không có vùng quá tải; kết quả không phụ thuộc thứ tự camera trong batch; chạy lại với `FrameClock` cho kết quả giống hệt.

9.2 Bộ dữ liệu và giao thức gán nhãn

1. **Nguồn:** clip tự quay ở Giai đoạn 1, có đồng ý bằng văn bản. Clip gốc chỉ lưu trên máy phát triển, mã hóa ổ đĩa, không đưa lên Git, xóa theo thời hạn đã cam kết khi xin đồng ý.
2. **Nhãn số đếm:** trích một khung mỗi 15 s; công cụ gán nhãn hiển thị các vùng; người gán nhãn đếm số người *thật sự đang ở* mỗi vùng theo phán đoán của mình, không theo điểm neo. Nhãn phải phản ánh thực tế, không phản ánh thuật toán.
3. **Nhãn bounding box:** trên một phần tư số khung đã trích, vẽ hộp cho mọi người, kèm `posture` và `occluded`, để đo recall detector.
4. **Nhãn phiên:** với vùng chỗ ngồi, đánh dấu thời điểm bắt đầu và kết thúc mỗi lần có người ngồi, độ chính xác 1 s, bằng cách tua video.
5. **Kiểm tra chất lượng:** 10% số khung được gán nhãn lần hai (người khác, hoặc chính mình sau một tuần). Nếu hai lần lệch nhau nhiều, làm rõ quy tắc gán nhãn rồi gán lại phần bị ảnh hưởng.
6. **Chia tập:** dev và test theo ngày quay. Tập test có danh sách file và mã băm trong repo, chỉ dùng để báo cáo.

> **Ước tính công gán nhãn**
> Với 6 giờ video: khoảng 1440 khung số đếm, khoảng 4 giờ công*; khoảng 360 khung bounding box, khoảng 6 giờ công*; nhãn phiên khoảng 6–8 giờ công*. Nên lên lịch phần này ngay trong Giai đoạn 1 vì Giai đoạn 2 phụ thuộc vào nó.

9.3 Chỉ số đánh giá

Gọi S là tập khung có nhãn, $n_z(s)$ là nhãn và $\hat{n}_z(s)$ là giá trị hệ thống tại khung s.

Chỉ số	Định nghĩa
MAE số đếm	$\frac{1}{\lvert S\rvert}\sum_s \lvert\hat{n}_z(s) - n_z(s)\rvert$, theo từng vùng và gộp theo nhóm; báo cáo thêm tách theo mức đông: vắng ($n \le 0{,}3\,C$), vừa, đông ($n > 0{,}7\,C$)
Tỉ lệ khớp chính xác	Tỉ lệ khung có $\hat{n}_z(s) = n_z(s)$
Độ lệch	$\frac{1}{\lvert S\rvert}\sum_s (\hat{n}_z(s) - n_z(s))$: dương là đếm thừa, âm là đếm thiếu. MAE không cho biết điều này
Sai số chỉ số sử dụng	$\lvert\hat{F}_z - F_z\rvert$ và $\lvert\hat{U}_z - U_z\rvert$ theo từng giờ, tính F, U từ nhãn với cùng công thức
Recall detector	TP / (TP + FN) ở IoU $\ge 0{,}5$, tách theo `posture` và `occluded`
Khớp phiên	Ghép phiên dự đoán với phiên thật bằng thuật toán Hungarian theo IoU thời gian, khớp khi IoU $\ge 0{,}5$; báo cáo precision, recall
Sai số đầu, cuối phiên	Trung vị và p90 của $\lvert\hat{t}_{\text{start}} - t_{\text{start}}\rvert$ và $\lvert\hat{t}_{\text{end}} - t_{\text{end}}\rvert$ trên các cặp đã khớp
Tỉ lệ phân mảnh	Số phiên dự đoán chồng lên một phiên thật, lấy trung bình; lý tưởng bằng 1
Đếm cửa	Ghép từng sự kiện với nhãn trong cửa sổ ± 1 s, cùng chiều; báo cáo precision và recall thay vì sai số tổng, vì đếm thừa và đếm thiếu có thể bù trừ

9.4 Giao thức benchmark

1. **Bước 1, so cấu hình ở 8 camera:** mô hình {n, s} × `imgsz` {640, 960} × backend {PyTorch, ORT CUDA FP32, ORT CUDA FP16}: 12 cấu hình.
2. **Bước 2, quét số camera:** với 2 cấu hình tốt nhất, chạy {1, 4, 8, 12, 16} camera: 10 cấu hình.
3. **Mỗi cấu hình:** khởi động 60 s không đo, đo 5 phút, lặp 3 lần, báo cáo trung vị. Tổng khoảng 6–7 giờ*, chạy qua đêm.
4. **Ghi lại:** số camera tối đa với mẫu bị bỏ $\le 1\%$; p50 và p95 của thời gian suy luận và độ tươi; GPU util, VRAM; **CPU cho decode**; RAM. Ở $f_s = 2$, giới hạn rất có thể nằm ở decode trên CPU chứ không ở GPU; đây là giả thuyết cần đo để xác nhận.

5. **Kết quả** lưu CSV kèm commit hash, cấu hình và thông số phần cứng; `make bench` sinh lại toàn bộ bảng.
6. **Kiểm tra độ chính xác** của mỗi backend trên tập test theo ngưỡng ở Giai đoạn 7.

9.5 Kiểm thử bơm lỗi

Kịch bản	Hành vi mong đợi
Dừng publisher của cam1 trong 30 s	cam1 chuyển `RECONNECTING`, vùng của cam1 thành `UNKNOWN`; camera khác không đổi; cam1 về `ONLINE` trong ≤ 10 s khi phát lại
Khởi động lại container MediaMTX	Mọi reader tự kết nối lại, không cần khởi động lại engine
Ngắt mạng một camera 90 s	Sau 60 s chuyển `OFFLINE` và cảnh báo; tự phục hồi khi có mạng
Nguồn nhanh hơn khả năng xử lý	Độ tươi không tăng dần theo thời gian; mẫu bị bỏ tăng và được ghi log, metric
Cấu hình sai (đa giác tự cắt, thiếu trường, `schema_version` lạ)	Camera đó ở trạng thái `ERROR` với thông báo rõ ràng; camera khác chạy bình thường
`kill -9` một reader	Supervisor khởi động lại trong ≤ 5 s; camera khác không bị ảnh hưởng
`kill -9` tiến trình suy luận	Khởi động lại; phiên đang mở được đóng với cờ `interrupted`; cơ sở dữ liệu không hỏng
Ổ đĩa đầy (volume nhỏ)	Metric lỗi ghi tăng, có cảnh báo, không lặp crash; tự ghi tiếp khi có chỗ trống
Khởi động lại container api	Dashboard tự kết nối lại WebSocket; engine không bị ảnh hưởng
Mất điện (tắt máy đột ngột)	Sau khi bật máy, mọi dịch vụ tự chạy; `PRAGMA integrity_check` trả `ok`
GPU không dùng được (lỗi driver)	Engine không sẵn sàng (`/ready` lỗi), cảnh báo rõ nguyên nhân; nếu cấu hình cho phép, chuyển sang CPU với ít camera hơn

9.6 Soak test

Chạy 24 giờ trên staging với 8 camera mô phỏng, 1 webcam và 1 camera thật. Mỗi giờ ghi RSS của từng tiến trình, VRAM, số file descriptor, kích thước cơ sở dữ liệu. Đạt khi không crash và RAM tăng dưới 10% so với mốc sau giờ đầu tiên (M7). Bộ nhớ tăng đều theo giờ là dấu hiệu rò rỉ, kể cả khi chưa vượt ngưỡng.

9.7 Kiểm thử quyền riêng tư

1. Chạy hệ thống 10 phút với profile `sim` và `OCC_ENV=production`.
2. Quét mọi volume và thay đổi hệ thống file của container (`docker diff`) tìm file có chữ ký ảnh hoặc video (JPEG, PNG, WebP, MP4, MKV). Đạt khi không có file nào.
3. Kiểm tra schema cơ sở dữ liệu không có cột BLOB và không có cột chứa `track`.
4. Kiểm tra log không chứa mật khẩu trong URL RTSP.
5. Kiểm tra cấu hình có `debug_save_frames: true` bị từ chối khi `OCC_ENV=production`.
6. Kiểm tra tĩnh: luật lint cấm `cv2.imwrite` và `cv2.VideoWriter` ngoài `occ/tools/`.

9.8 Nghiệm thu tại hiện trường

Sau khi cài ở một địa điểm mới, trước khi bàn giao:

1. Ba lần đếm tay, mỗi lần 30 phút, ở ba mức đông khác nhau: cứ 2 phút đếm số người trong từng không gian và so với dashboard (15 mẫu mỗi lần).
2. Đạt khi MAE nằm trong ngưỡng M1 cộng biên 0,3 người (đếm tay tại chỗ cũng có sai số).

3. Rút dây một camera: cảnh báo xuất hiện, dashboard hiển thị `UNKNOWN`, cắm lại thì tự phục hồi.
4. Kiểm tra bản sao lưu đầu tiên đã được tạo và phục hồi thử được.
5. Biên bản nghiệm thu: phiên bản, số camera, số vùng, kết quả đếm tay, người phụ trách địa điểm ký xác nhận.

CHƯƠNG 10

Triển khai và vận hành

10.1 Yêu cầu phần cứng và phần mềm

Hạng mục	Tối thiểu (1–2 camera, chỉ CPU)	Khuyến nghị (8 camera trở lên)
CPU	4 nhân x86-64	8 nhân trở lên (decode video tốn CPU)*
RAM	8 GB	16 GB
GPU	Không cần	NVIDIA, tối thiểu 6 GB VRAM*
Lưu trữ	SSD 128 GB	SSD 256 GB trở lên
Mạng	Ethernet 100 Mbit/s	Ethernet gigabit; switch PoE cho camera
Nguồn	—	UPS cho PC và switch
Hệ điều hành	Ubuntu 22.04 hoặc 24.04 LTS	Ubuntu 24.04 LTS
Phần mềm	Docker Engine với Compose plugin	Thêm driver NVIDIA và NVIDIA Container Toolkit, phiên bản theo bảng tương thích của CUDA
Camera	Camera IP có RTSP, H.264	Thêm luồng phụ 720p ở 5–10 FPS; gắn cao 2,5–3 m, nghiêng 30–45°*, tránh ngược sáng

Cấu hình khuyến nghị là ước tính ban đầu; con số thật lấy từ benchmark ở Giai đoạn 7.

10.2 Mô hình mạng

- Camera nằm trong VLAN riêng, không ra Internet. Chỉ PC xử lý truy cập được camera.
- Dashboard và API phục vụ trong LAN qua Caddy với HTTPS (profile `proxy`); chỉ mở cổng 443 ra LAN.
- Cổng RTSP 8554 và cổng metric chỉ dùng trong mạng Docker, không mở ra ngoài.
- Sau khi cài đặt, hệ thống không cần Internet. Với nơi không có Internet, dùng gói cài offline.

10.3 Các bước triển khai lần đầu

1. **Khảo sát:** danh sách không gian, sức chứa, giờ mở cửa; vị trí lắp camera để thấy chỗ ngồi theo góc nghiêng; nguồn điện, mạng.
2. **Pháp lý và thông báo:** đồng ý của tổ chức; biển thông báo ở mọi lối vào khu có camera (Chương 11).
3. **Lắp camera:** cấu hình luồng phụ, đặt mật khẩu riêng cho từng camera, đặt IP tĩnh hoặc giữ DHCP cố định.
4. **Chuẩn bị PC:** cài Ubuntu, driver NVIDIA, Docker, NVIDIA Container Toolkit; chạy `nvidia-smi` trong một container để kiểm tra.
5. **Cài phần mềm:** giải nén gói triển khai, chạy `sudo ./install.sh`. Script kiểm tra môi trường, tự sinh bí mật vào `.env`, kéo hoặc nạp image, chạy migration, khởi động và chờ `/ready`.
6. **Khai báo site:** sửa `configs/site.yaml` (không gian, sức chứa, giờ mở cửa, múi giờ); chạy `occ config validate`.
7. **Thêm camera:** thêm nguồn vào cấu hình MediaMTX và file camera; kiểm tra camera `ONLINE` trên trang sức khỏe hệ thống.
8. **Vẽ vùng:** dùng công cụ vẽ vùng cho từng camera; thêm vùng loại trừ cho poster, màn hình, gương.
9. **Kiểm tra phòng trống:** khi không có ai, mọi vùng phải bằng 0 trong 10 phút. Nếu không, thêm vùng loại trừ.
10. **Nghiệm thu:** theo mục 9.8.

11. **Bật vận hành:** profile `monitoring`, kênh nhận cảnh báo (email hoặc webhook), kiểm tra bản sao lưu đầu tiên.
12. **Bàn giao:** tài khoản quản trị, token, runbook, biên bản nghiệm thu.

10.4 Thay đổi thường gặp

Việc	Cách làm
Thêm camera	Thêm nguồn trong MediaMTX và file camera; engine tự phát hiện file mới, không cần khởi động lại
Sửa vùng	Công cụ vẽ vùng; bản cũ giữ trong `configs/.history/`; số liệu trước thời điểm sửa không bị tính lại
Đổi giờ mở cửa	Sửa `site.yaml`; áp dụng khi truy vấn, nên báo cáo cũ cũng theo giờ mới; ghi lại thời điểm đổi trong nhật ký thay đổi
Đổi mô hình	Đặt file ONNX mới vào volume `models`, sửa `engine.model`, khởi động lại engine; chạy `make eval` trên tập test trước khi đổi ở production
Nâng cấp phần mềm	`./deploy.sh --version X.Y.Z` (mục 8.6)

> **Sửa vùng làm số liệu không còn so sánh được**
> Khi một vùng bị vẽ lại, số liệu trước và sau thời điểm đó đo trên hai định nghĩa khác nhau. Dashboard đánh dấu thời điểm sửa trên biểu đồ, và báo cáo so sánh qua mốc đó phải ghi chú.

10.5 Giám sát và cảnh báo

Cảnh báo	Điều kiện	Mức	Hành động đầu tiên
CameraOffline	`occ_camera_state` = 0 trong 5 phút	cao	Kiểm tra nguồn, mạng, đăng nhập camera
CameraConfigError	`occ_camera_state` = 3	cao	Đọc log của camera; chạy `occ config validate`
SamplesDropped	Tỉ lệ mẫu bị bỏ $> 5\%$ trong 10 phút	trung bình	Kiểm tra tải GPU và CPU; giảm f_s hoặc số camera
FreshnessHigh	p95 `occ_freshness_seconds` > 5 s trong 10 phút	trung bình	Kiểm tra writer và cơ sở dữ liệu
EngineNotReady	`/ready` của engine lỗi trong 2 phút	cao	Xem log engine; kiểm tra GPU bằng `nvidia-smi`
DbWriteErrors	`occ_db_write_errors_total` tăng	cao	Kiểm tra ổ đĩa, quyền ghi, `integrity_check`
DiskAlmostFull	Ổ dữ liệu dùng $> 80\%$	trung bình	Kiểm tra chính sách lưu giữ, dọn image cũ
BackupMissing	Không có bản sao lưu mới trong 26 giờ	trung bình	Xem log dịch vụ sao lưu
ZoneStuck	Một vùng giữ nguyên giá trị khác 0 suốt ngoài giờ mở cửa	thấp	Có thể là false positive cố định; thêm vùng loại trừ

10.6 Sao lưu và phục hồi

- Dịch vụ `backup` chạy hằng đêm, dùng online backup API của SQLite nên an toàn khi writer đang ghi. Giữ 14 bản; sao lưu cả thư mục cấu hình.
- Nên chép bản sao lưu sang một ổ khác hoặc máy khác trong LAN. Bản sao lưu chỉ chứa số đếm, không chứa hình ảnh.
- Phục hồi: `./restore.sh <file>` dừng engine và api, kiểm tra `integrity_check` của bản sao lưu, thay file, khởi động lại.
- Diễn tập phục hồi trên staging trước mỗi bản phát hành và mỗi quý ở production.

10.7 Runbook sự cố

Triệu chứng	Kiểm tra	Xử lý
Mọi camera `RECONNECTING`	`docker compose ps`: MediaMTX còn chạy?	Khởi động lại MediaMTX; reader tự kết nối lại
Một camera `OFFLINE`	Ping camera; mở RTSP bằng `ffprobe` từ PC	Sửa mạng hoặc mật khẩu; cập nhật `.env` nếu đổi mật khẩu
Số người luôn lớn hơn 0 khi phòng trống	Ảnh chụp của công cụ vẽ vùng có thấy poster hoặc màn hình?	Thêm vùng loại trừ
Đếm thiếu khi đông	So recall theo mức đông trong báo cáo đánh giá	Đổi góc camera cao hơn; thử mô hình lớn hơn hoặc `imgsz` lớn hơn
Engine lặp khởi động lại	`docker compose logs engine`; `nvidia-smi` trong container	Sửa driver hoặc toolkit; tạm chuyển backend CPU
Dashboard không cập nhật	`/ready` của api; WebSocket trong công cụ dev của trình duyệt	Khởi động lại api; kiểm tra Caddy
Sau nâng cấp bị lỗi	Log của `deploy.sh`; ghi chú tương thích trong CHANGELOG	`./rollback.sh`

CHƯƠNG 11

Quyền riêng tư, bảo mật và giấy phép

11.1 Nguyên tắc

1. **Thu thập tối thiểu:** chỉ lưu số đếm theo vùng và phiên không gắn danh tính. Không lưu khung hình, không lưu `track_id`, không nhận dạng khuôn mặt.
2. **Xử lý tại chỗ:** khung hình được xử lý trên PC tại địa điểm, không gửi lên cloud.
3. **Giới hạn mục đích:** dữ liệu chỉ dùng để đo mức sử dụng không gian. Hệ thống không được dùng để chấm công, đánh giá hiệu suất hay kỷ luật cá nhân; README và tài liệu bàn giao ghi rõ điều này.
4. **Minh bạch:** biển thông báo ở mọi lối vào khu có camera; trang công khai giải thích hệ thống đo gì và không đo gì.
5. **Lưu giữ có thời hạn:** theo bảng lưu giữ ở mục 5.3.

11.2 Kiểm kê dữ liệu

Dữ liệu	Nơi lưu	Thời hạn	Ai truy cập
Khung hình	RAM của engine	Vài giây	Không ai; không ghi ra đĩa
Ảnh chụp vẽ vùng (đã làm mờ người)	RAM của api	10 phút	Quản trị viên; có log kiểm toán
Số đếm theo vùng	SQLite	30 hoặc 400 ngày	Người có token `read`
Phiên chỗ ngồi	SQLite	400 ngày	Người có token `read`
Log hệ thống	Log Docker	Xoay vòng theo dung lượng	Quản trị viên
Clip phục vụ phát triển	Máy phát triển, ổ mã hóa	Theo cam kết khi xin đồng ý	Nhóm phát triển

11.3 Thông báo và đồng ý

Mẫu biển thông báo

Khu vực có camera đo mức sử dụng không gian. Camera dùng để đếm số người trong từng khu vực nhằm bố trí chỗ ngồi hợp lý hơn. Hệ thống không ghi hình, không lưu hình ảnh và không nhận dạng bất kỳ ai; chỉ lưu số người theo khu vực và thời gian. Thông tin thêm và liên hệ: [đơn vị phụ trách, email].

Với clip quay để phát triển và đánh giá, cần đồng ý bằng văn bản của người xuất hiện trong clip, nêu rõ mục đích, nơi lưu, thời hạn xóa, và việc video demo công khai sẽ được làm mờ.

Pháp lý

Hình ảnh người có thể nhận diện được là dữ liệu cá nhân. Việt Nam có quy định về bảo vệ dữ liệu cá nhân, và tổ chức nơi lắp đặt có thể có quy định nội bộ riêng. Trước khi triển khai thật, cần đối chiếu văn bản pháp luật đang có hiệu lực tại thời điểm đó và quy định của tổ chức. Tài liệu này không phải tư vấn pháp lý.

11.4 Bảo mật

- Bí mật (mật khẩu camera, token, mật khẩu quản trị) chỉ nằm trong `.env` trên máy đích, do `install.sh` sinh; `.env` không bao giờ vào Git.
- Không có mật khẩu mặc định. Token có phạm vi `read` hoặc `admin`; thu hồi được.
- Container chạy bằng user không phải root; hệ thống file gốc chỉ đọc khi có thể; volume cấu hình chỉ đọc với engine.
- Camera trong VLAN riêng, mật khẩu riêng cho từng camera, tắt các dịch vụ không dùng trên camera.
- Phụ thuộc được ghim phiên bản (`uv.lock`, tag image cụ thể); CI quét bí mật và lỗ hổng image.

11.5 Giấy phép

Thành phần	Giấy phép	Ghi chú
Ultralytics YOLO (mã và trọng số)	AGPL-3.0	Dùng cho portfolio mã nguồn mở là phù hợp. Sản phẩm thương mại đóng cần giấy phép Enterprise hoặc mô hình có giấy phép thoáng hơn (ví dụ YOLOX, Apache-2.0). Chỉ chạy bằng ONNX Runtime không làm trọng số thoát khỏi giấy phép này
ByteTrack	MIT	
ONNX Runtime	MIT	
OpenCV	Apache-2.0	Phiên bản 4.5 trở lên
FFmpeg	LGPL hoặc GPL	Tùy cách build; kiểm tra bản dùng trong image
MediaMTX	MIT	
FastAPI	MIT	
SQLite	Public domain	
Prometheus, Caddy	Apache-2.0	
Grafana	AGPL-3.0	Dùng nguyên bản không sửa đổi trong profile tùy chọn
Bộ dữ liệu công khai	Theo từng bộ	Nhiều bộ chỉ cho phép dùng phi thương mại; kiểm tra trước khi dùng

Giấy phép có thể thay đổi giữa các phiên bản; kiểm tra lại file LICENSE của đúng phiên bản đang dùng.

CHƯƠNG 12

Rủi ro và giảm thiểu

Rủi ro	Mức	Giảm thiểu
Detector bỏ sót nhiều người ngồi hoặc nhìn từ trên	cao	Phép thử khả thi ở tuần đầu (mục 2.1); đổi góc camera; mô hình lớn hơn, `imgsz` lớn hơn, tiling; dự phòng mô hình phát hiện đầu người hoặc fine-tune
Docker không thấy GPU	cao	Kiểm tra ngày đầu của Giai đoạn 0; ghim phiên bản driver, CUDA, cuDNN, ONNX Runtime theo bảng tương thích chính thức
Không có địa điểm quay hoặc pilot	cao	Xin phép sớm, song song Giai đoạn 0; dự phòng phòng của bản thân, nhóm bạn có đồng ý
Công gán nhãn nhiều hơn dự kiến	trung bình	Lấy mẫu 15 s thay vì gán nhãn liên tục; bắt đầu ngay Giai đoạn 1; giảm số giờ video trước khi giảm chất lượng nhãn
Tracker đổi ID làm vỡ phiên	trung bình	Phiên theo trạng thái chỗ ngồi; nối track; báo cáo tỉ lệ phân mảnh
False positive cố định (poster, màn hình)	trung bình	Vùng loại trừ; kiểm tra phòng trống; cảnh báo `ZoneStuck`
Nhiều camera nhìn chồng một không gian	trung bình	Vùng không chồng lấn trên mặt bằng; kiểm tra cấu hình cảnh báo
Decode trên CPU là nút thắt	trung bình	Dùng luồng phụ độ phân giải thấp, FPS thấp; đo CPU trong benchmark; decode bằng phần cứng nếu cần
SQLite không đủ khi mở rộng	thấp	Lớp lưu trữ qua SQLAlchemy Core; chuyển PostgreSQL khi cần
Người bị quay phản đối	trung bình	Thông báo rõ ràng; không ghi hình; giới hạn mục đích; trang giải thích công khai
Phạm vi phình to	trung bình	Cờ tính năng; thứ tự cắt giảm ở Chương 7; đóng băng tính năng ở Giai đoạn 7

CHƯƠNG 13

Definition of Done cho v1.0

☐ Phép thử khả thi đã chạy; quyết định và số đo được ghi trong ADR.

☐ Tập test đóng băng; báo cáo M1–M3 trên tập test, tách theo mức đông, kèm độ lệch và độ phủ dữ liệu.

☐ Đạt M4–M9, hoặc có giải thích bằng số đo cho mục chưa đạt.

☐ Mọi kịch bản bơm lỗi ở mục 9.5 đạt; soak 24 giờ đạt.

☐ Kiểm thử quyền riêng tư đạt; không có ảnh hoặc video trên đĩa ở chế độ production.

☐ CI xanh; độ phủ test của `analytics` $\geq 90\%$; hồi quy độ chính xác chạy trong CI.

☐ Báo cáo benchmark tái lập được bằng `make bench` (CSV, script, thông số phần cứng).

☐ Cài mới bằng `install.sh`, nâng cấp bằng `deploy.sh`, rollback bằng `rollback.sh` đều đã diễn tập; diễn tập phục hồi đạt.

☐ Pilot 7 ngày ở địa điểm thật; nghiệm thu tại hiện trường đạt; có biên bản.

☐ Tài liệu: README (cài đặt, cấu hình, giới hạn, định nghĩa chỉ số), `deployment.md`, `operations.md`, `privacy.md`, các ADR, OpenAPI.

☐ GitHub Release v1.0.0 kèm gói triển khai, CHANGELOG, video demo 1–2 phút đã làm mờ người.

PHỤ LỤC A

Cấu trúc repository

Mã nguồn A.1: Cấu trúc đề xuất

```
indoor-occupancy/
  pyproject.toml  uv.lock  Makefile  docker-bake.hcl
  docker-compose.yml  docker-compose.dev.yml  docker-compose.ci.yml  .env.example
  README.md  CHANGELOG.md  LICENSE
  .github/workflows/ci.yml  nightly.yml
  configs/
    schema/                   # JSON Schema generated from Pydantic models
    examples/site.yaml  examples/cameras/*.yaml  mediamtx.yml
  deploy/
    install.sh  deploy.sh  rollback.sh  restore.sh  make_bundle.sh
    caddy/Caddyfile  prometheus/  grafana/
  docker/
    Dockerfile.engine  Dockerfile.api  Dockerfile.tools
  src/occ/
    common/      config.py  clock.py  logging.py  ids.py  health.py
    reader/      rtsp.py  sampler.py  shm.py  backoff.py
    engine/      supervisor.py  batcher.py  detector_ort.py  detector_torch.py
                 postprocess.py  pipeline.py  snapshot.py
    analytics/   filters.py  assign.py  smoothing.py  seat_state.py
                 tracking.py  stitching.py  door.py  aggregate.py
    storage/     tables.py  writer.py  retention.py  backup.py  migrations/
    api/         main.py  auth.py  ws.py  routes/  schemas.py
    dashboard/   templates/  static/          # vendored JS, no CDN
    tools/       simulate.py  sample_frames.py  cache_detections.py
                 eval_counts.py  eval_sessions.py  onnx_fp16.py
    cli.py                       # `occ` command
  tests/
    unit/  property/  integration/  e2e/  chaos/  soak/  privacy/
    fixtures/    detections/  baseline.json  sample.db
  bench/         run_bench.py  matrix.yaml  results/
  docs/
    architecture.md  deployment.md  operations.md  privacy.md  targets.md
    adr/0001-record-architecture-decisions.md ...
  data/          # gitignored: clips/, labels/, cache/
```

PHỤ LỤC B

File Compose mẫu

Tag image lấy từ `.env` (`OCC_VERSION`, `MEDIAMTX_TAG`, ...) để mỗi lần deploy ghim đúng phiên bản.

Mã nguồn B.1: `docker-compose.yml`

```
name: occupancy

x-common: &common
  restart: unless-stopped
  env_file: .env
  networks: [internal]
  logging:
    driver: json-file
    options: {max-size: "20m", max-file: "5"}

services:
  mediamtx:
    <<: *common
    image: bluenviron/mediamtx:${MEDIAMTX_TAG}
    volumes: ["./configs/mediamtx.yml:/mediamtx.yml:ro"]

  sim-publisher:                      # dev, CI, staging only
    <<: *common
    profiles: ["sim"]
    image: ${REGISTRY}/occ-tools:${OCC_VERSION}
    command: ["occ", "tools", "simulate", "--clips", "/clips",
              "--rtsp", "rtsp://mediamtx:8554"]
    volumes: ["./data/clips:/clips:ro"]
    depends_on: [mediamtx]

  webcam-publisher:                   # Linux only; on Windows run FFmpeg (dshow) on the host
    <<: *common
    profiles: ["webcam"]
    image: ${REGISTRY}/occ-tools:${OCC_VERSION}
    devices: ["/dev/video0:/dev/video0"]
    command: ["ffmpeg", "-f", "v4l2", "-framerate", "10", "-video_size", "1280x720",
              "-i", "/dev/video0", "-c:v", "libx264", "-preset", "ultrafast",
              "-tune", "zerolatency", "-g", "20", "-f", "rtsp", "rtsp://mediamtx:8554/cam0"]
    depends_on: [mediamtx]

  engine:
    <<: *common
    image: ${REGISTRY}/occ-engine:${OCC_VERSION}
    environment: {OCC_ENV: "${OCC_ENV:-production}"}
    volumes:
      - ./configs:/configs:ro
      - ./models:/models:ro
      - occ-data:/data
    shm_size: "1gb"
    deploy:
      resources:
        reservations:
          devices: [{driver: nvidia, count: 1, capabilities: [gpu]}]
    healthcheck:
      test: ["CMD", "occ", "health", "http://localhost:9100/health"]
      interval: 15s
      timeout: 5s
      retries: 4
      start_period: 60s
```

```
    depends_on: [mediamtx]

  api:
    <<: *common
    image: ${REGISTRY}/occ-api:${OCC_VERSION}
    volumes:
      - ./configs:/configs            # zone editor writes camera files here
      - occ-data:/data                # opened read-only; WAL readers still need the -shm file
    ports: ["${API_BIND:-127.0.0.1}:8000:8000"]
    healthcheck:
      test: ["CMD", "occ", "health", "http://localhost:8000/health"]
      interval: 15s
      timeout: 5s
      retries: 4
    depends_on:
      engine: {condition: service_healthy}

  backup:
    <<: *common
    image: ${REGISTRY}/occ-tools:${OCC_VERSION}
    command: ["occ", "backup", "run", "--every", "24h", "--keep", "14"]
    volumes: ["occ-data:/data", "./backups:/backups", "./configs:/configs:ro"]

  proxy:
    <<: *common
    profiles: ["proxy"]
    image: caddy:${CADDY_TAG}
    ports: ["443:443"]
    volumes: ["./deploy/caddy/Caddyfile:/etc/caddy/Caddyfile:ro", "caddy-data:/data"]
    depends_on: [api]

  prometheus:
    <<: *common
    profiles: ["monitoring"]
    image: prom/prometheus:${PROMETHEUS_TAG}
    volumes: ["./deploy/prometheus:/etc/prometheus:ro", "prom-data:/prometheus"]

  grafana:
    <<: *common
    profiles: ["monitoring"]
    image: grafana/grafana:${GRAFANA_TAG}
    ports: ["${API_BIND:-127.0.0.1}:3000:3000"]
    volumes: ["./deploy/grafana:/etc/grafana/provisioning:ro", "grafana-data:/var/lib/grafana"]

volumes: {occ-data: {}, caddy-data: {}, prom-data: {}, grafana-data: {}}
networks: {internal: {}}
```

File `docker-compose.dev.yml` ghi đè engine: bỏ phần đặt trước GPU, đặt `engine.backend: ort-cpu` và mô hình cỡ n, để chạy được trên laptop.

PHỤ LỤC C

Dockerfile của engine

Mã nguồn C.1: `docker/Dockerfile.engine`. Tag CUDA và phiên bản uv là ví dụ; đối chiếu bảng tương thích của ONNX Runtime trước khi chọn.

```
# syntax=docker/dockerfile:1
ARG CUDA_TAG=12.4.1-cudnn-runtime-ubuntu22.04
FROM nvidia/cuda:${CUDA_TAG}
ENV DEBIAN_FRONTEND=noninteractive PYTHONUNBUFFERED=1 UV_COMPILE_BYTECODE=1
RUN apt-get update \
 && apt-get install -y --no-install-recommends python3 python3-venv ca-certificates \
 && rm -rf /var/lib/apt/lists/*
COPY --from=ghcr.io/astral-sh/uv:0.5.11 /uv /usr/local/bin/uv
WORKDIR /app
# dependencies first for layer caching; the "engine" extra has onnxruntime-gpu,
# opencv-python-headless, numpy, shapely, a standalone ByteTrack - no torch, no ultralytics
COPY pyproject.toml uv.lock ./
RUN uv sync --frozen --no-dev --extra engine --no-install-project
COPY src ./src
RUN uv sync --frozen --no-dev --extra engine
RUN useradd --uid 10001 --create-home occ && mkdir -p /data && chown occ /data
USER occ
ENV PATH="/app/.venv/bin:${PATH}"
EXPOSE 9100
ENTRYPOINT ["occ", "engine", "run", "--config-dir", "/configs"]
```

PHỤ LỤC D

Makefile

Mã nguồn D.1: `Makefile` (các lệnh chính)

```
VERSION  ?= $(shell git describe --tags --always --dirty | sed 's/^v//')
REGISTRY ?= ghcr.io/your-user
COMPOSE  := docker compose --env-file .env

setup:        ## dev environment
    uv sync --all-extras && uv run pre-commit install
lint:         ## ruff + mypy
    uv run ruff check . && uv run ruff format --check . && uv run mypy src
test:         ## unit + property + integration
    uv run pytest tests/unit tests/property tests/integration -q \
      --cov=occ.analytics --cov-fail-under=90
eval:         ## accuracy report from cached detections (no GPU)
    uv run occ eval --split test --detections data/cache --out reports/eval.md
bench:        ## benchmark matrix (GPU, several hours)
    uv run python bench/run_bench.py --matrix bench/matrix.yaml --out bench/results
chaos:        ## fault injection against a running stack
    uv run pytest tests/chaos -q
soak:         ## 24 h soak test
    uv run python tests/soak/run_soak.py --hours 24
build:
    OCC_VERSION=$(VERSION) REGISTRY=$(REGISTRY) docker buildx bake
up-dev:       ## whole stack on a laptop: CPU backend + simulated cameras
    OCC_ENV=dev $(COMPOSE) -f docker-compose.yml -f docker-compose.dev.yml --profile sim up -d --build
down:
    $(COMPOSE) down
logs:
    $(COMPOSE) logs -f --tail=200
deploy:
    ./deploy/deploy.sh --version $(VERSION)
rollback:
    ./deploy/rollback.sh
export-model: ## YOLO -> ONNX with dynamic batch, then FP16 copy (tools image has ultralytics)
    docker run --rm --gpus all -v $(PWD)/models:/models $(REGISTRY)/occ-tools:$(VERSION) \
      sh -c "yolo export model=/models/yolo11s.pt format=onnx dynamic=True imgsz=640 && \
             occ tools onnx-fp16 /models/yolo11s.onnx /models/yolo11s-fp16.onnx"

.PHONY: setup lint test eval bench chaos soak build up-dev down logs deploy rollback export-model
```

PHỤ LỤC E

Workflow CI

Mã nguồn E.1: `.github/workflows/ci.yml` (rút gọn)

```
name: ci
on:
  pull_request:
  push:
    branches: [main]
    tags: ["v*.*.*"]

jobs:
  check:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - run: uv sync --all-extras --frozen
      - run: make lint
      - run: make test
      - run: uv run occ eval --split fixture --detections tests/fixtures/detections
               --check-baseline tests/fixtures/baseline.json

  e2e:
    needs: check
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: astral-sh/setup-uv@v5
      - uses: docker/setup-buildx-action@v3
      - run: uv sync --all-extras --frozen
      - run: docker buildx bake --load ci
      - run: OCC_ENV=production docker compose -f docker-compose.yml
               -f docker-compose.ci.yml --profile sim up -d
      - run: uv run pytest tests/e2e -q        # /ready, zone_live changes, API contract
      - run: uv run pytest tests/privacy -q    # no image/video files, no BLOB, no secrets in logs
      - if: always()
        run: docker compose logs --no-color > compose.log

  publish:
    needs: e2e
    if: github.event_name == 'push'
    runs-on: ubuntu-24.04
    permissions: {contents: read, packages: write}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/setup-buildx-action@v3
      - uses: docker/login-action@v3
        with: {registry: ghcr.io, username: "${{ github.actor }}",
               password: "${{ secrets.GITHUB_TOKEN }}"}
      - name: version
        run: |
          if [[ "$GITHUB_REF" == refs/tags/v* ]]; then
            echo "OCC_VERSION=${GITHUB_REF_NAME#v}" >> "$GITHUB_ENV"
          else
            echo "OCC_VERSION=sha-${GITHUB_SHA::7}" >> "$GITHUB_ENV"
          fi
      - run: docker buildx bake --push release
```

```
  release:
    needs: publish
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-24.04
    permissions: {contents: write}
    steps:
      - uses: actions/checkout@v4
      - run: ./deploy/make_bundle.sh "${GITHUB_REF_NAME#v}"
      - run: gh release create "$GITHUB_REF_NAME" dist/*.tar.gz --notes-file dist/RELEASE_NOTES.md
        env: {GH_TOKEN: "${{ secrets.GITHUB_TOKEN }}"}
```

PHỤ LỤC F

Tài liệu tham khảo

[1] Ultralytics, *Export* và *Multi-Object Tracking*. https://docs.ultralytics.com/modes/export/, https://docs.ultralytics.com/modes/track/

[2] Y. Zhang và cộng sự, *ByteTrack: Multi-Object Tracking by Associating Every Detection Box*, ECCV 2022. https://github.com/ifzhang/ByteTrack

[3] Roboflow, *supervision* (có cài đặt ByteTrack độc lập). https://github.com/roboflow/supervision

[4] ONNX Runtime, *CUDA Execution Provider*. https://onnxruntime.ai/docs/execution-providers/CUDA-ExecutionProvider.html

[5] MediaMTX. https://github.com/bluenviron/mediamtx; FFmpeg documentation. https://ffmpeg.org/ffmpeg.html

[6] SQLite, *Write-Ahead Logging* và *Online Backup API*. https://www.sqlite.org/wal.html, https://www.sqlite.org/backup.html

[7] Alembic. https://alembic.sqlalchemy.org/; Shapely. https://shapely.readthedocs.io/

[8] FastAPI. https://fastapi.tiangolo.com/

[9] Docker Compose. https://docs.docker.com/compose/; NVIDIA Container Toolkit, *Installation Guide*. https://docs.nvidia.com/datacenter/cloud-native/container-toolkit/latest/install-guide.html

[10] Prometheus. https://prometheus.io/docs/

[11] Hypothesis. https://hypothesis.readthedocs.io/; Schemathesis. https://schemathesis.readthedocs.io/

[12] CVAT, công cụ gán nhãn ảnh và video. https://github.com/cvat-ai/cvat

[13] uv. https://docs.astral.sh/uv/

[14] Semantic Versioning. https://semver.org/; Conventional Commits. https://www.conventionalcommits.org/